

Annexes rédigées de la présentation

Yannis ABDELOUAHAB - n°21691

Session 2026

Table des matières

1	Coordonnées des Points de Lagrange	2
1.1	Définition du système	2
1.2	Mouvement relatif des masses M_1 et M_2	3
1.3	Expression du potentiel effectif dans R_{NG}	3
1.4	Coordonnées des points colinéaires	4
1.4.1	Coordonnées de L1	5
1.4.2	Coordonnées de L2	6
1.4.3	Coordonnées de L3	6
1.5	Coordonnées des points équilatéraux	6
1.6	Représentation des points de Lagrange	7
2	Stabilité des Points de Lagrange	9
2.1	Linéarisation des équations du mouvement	9
2.2	Stabilité de L1 et L2	10
2.2.1	Calcul explicite approché du linéarisé	10
2.2.2	Réduction du linéarisé sur les sous-espaces stables réels	11
2.2.3	Calcul de la matrice de passage	12
2.2.4	Calcul des solutions du linéarisé	12
3	Approche asymptotique des orbites autour de L1 et L2	14
3.1	Existence d'orbites périodiques	14
3.2	Introduction aux variétés d'une orbite périodique	14
3.3	La matrice de Monodromie	14
3.4	Lien avec le flot	16
3.5	Étude théorique de la matrice de Monodromie	16
3.5.1	Première multiplicité de 1 comme valeur propre	16
3.5.2	Deuxième multiplicité de 1 comme valeur propre	17
3.6	Etude numérique de la matrice de Monodromie	17
3.6.1	Calcul d'une orbite	18

4	Listings Python	20
4.1	"Hub centrale" de la base de codes	20
4.2	Codes d'implémentations de méthodes d'intégration numérique	23
4.2.1	Implémentation de Euler Implicite	23
4.2.2	Implémentation de Runge-Kutta	25
4.2.3	Implémentation de Runge-Kutta 4	28
4.2.4	Implémentation de Verlet	31
4.2.5	Comparaison de performance de différents schémas	34
4.3	Méthodes d'affichage	37
4.4	Affichage du potentiel apparent sur les fenêtres de trajectoires	49
4.5	Visualisations du potentiel apparent	51
4.6	Codes utilitaires	57
4.6.1	Calcul des écarts relatifs	57
4.6.2	Calcul de l'énergie	57
4.6.3	Calcul de l'espace de phase	58
4.7	Calcul d'une condition initiale pour le système non-linéaire	60
4.8	Calcul du linéarisé en tout point d'une trajectoire	63
4.9	Calcul de la matrice de monodromie et obtention des éléments propres	65

Résumé

Le problème à trois corps restreint circulaire planaire (PCR3BP en anglais) est l'étude du mouvement relatif d'une des masses, supposée négligeable devant les deux autres. Nous chercherons à déterminer les coordonnées des points de Lagrange, qui sont cinq points d'équilibre du PCR3BP. Nous mènerons ensuite une étude de leur stabilité. A terme, nous chercherons à déterminer des orbites périodiques autour des points L1 et L2 en particulier, ainsi que mener une recherche de trajectoires de transfert qui ferait utilisation des variétés stables et instables liées aux points de Lagrange.

1 Coordonnées des Points de Lagrange

1.1 Définition du système

Nous étudions le système *satellite de masse m* que l'on associe au point matériel M . On se place dans le référentiel galiléen – que l'on note R_G – qui a pour centre le barycentre des deux masses M_1 et M_2 qu'on confondra avec le point matériel auquel leur astre respectif est associé.

On montre dans la sous-partie suivante que M_1 et M_2 sont en orbite parfaitement circulaire autour du barycentre, de pulsation de rotation identique ω constante. Dans le cadre de cette étude, nous considérerons le mouvement plan (à noter qu'il est simple de montrer qu'il n'existe aucun point d'équilibre en dehors du plan $Z = 0$).

On cherche à exprimer les coordonnées de M dans le référentiel *non-galiléen* – R_{NG} – en rotation dans le référentiel galiléen R_G pour simplifier les équations du mouvement. R_{NG} est alors défini comme le référentiel dans lequel M_1 et M_2 sont immobiles (cf. figure 1). On mènera la majorité de notre étude dans ce référentiel.

1.2 Mouvement relatif des masses M_1 et M_2

cf. les annexes du fichier pdf de la présentation.

1.3 Expression du potentiel effectif dans R_{NG}

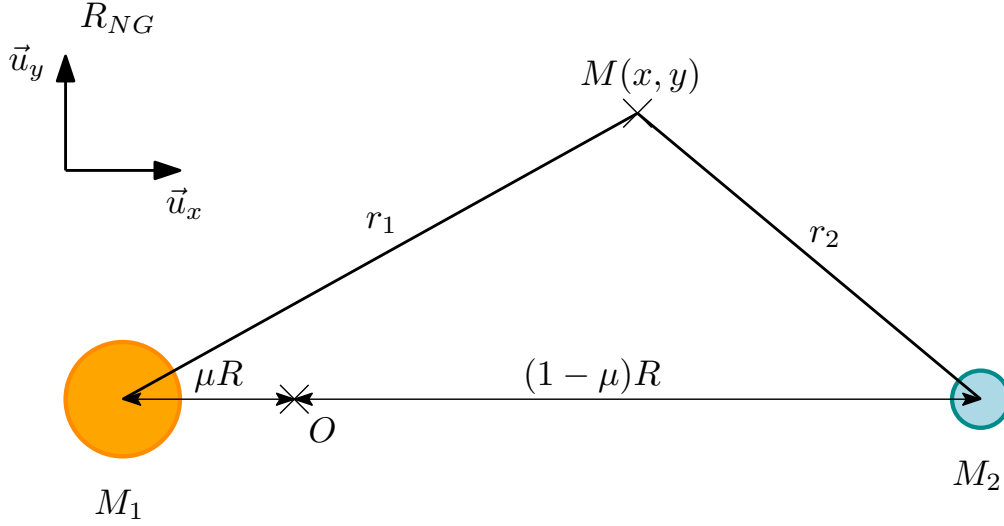


FIGURE 1 – Schéma du référentiel R_{NG} .

On cherche à déterminer les coordonnées d'un point (x, y) dans R_{NG} défini comme ci-dessus dans la figure 1. On note $(\vec{e}_x, \vec{e}_y)$ une base de R_G et $(\vec{u}_x, \vec{u}_y)$ une base de R_{NG} , les deux en coordonnées cartésiennes. On note α l'angle entre R_G et R_{NG} , ainsi que ω la vitesse angulaire de R_{NG} par rapport à R_G , que l'on suppose constante. Notons U l'énergie potentielle associée à M_1 et M_2 . On a alors dans le référentiel galiléen $\vec{F} = -\vec{\nabla}U$ où $\vec{F}$ est la résultante des forces de gravitation, ce qui se traduit par le système :

$$\begin{cases} m\ddot{X} = -\frac{\partial U}{\partial X} =: \mathcal{U}_X \\ m\ddot{Y} = -\frac{\partial U}{\partial Y} =: \mathcal{U}_Y \end{cases} \quad (1)$$

où (X, Y) sont les coordonnées d'un point P dans R_G . On note (x, y) ses coordonnées dans R_{NG} . C'est alors l'image de (X, Y) par une certaine rotation, à savoir A_t^{-1} (A_t étant défini juste après) :

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{pmatrix} \begin{pmatrix} X \\ Y \end{pmatrix} \quad (2)$$

C'est une matrice inversible dont l'inverse est $\begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix} =: A_t$. Cette dernière vérifie donc l'équation $\begin{pmatrix} X \\ Y \end{pmatrix} = A_t \begin{pmatrix} x \\ y \end{pmatrix}$.

On cherche maintenant à calculer $\begin{pmatrix} \ddot{X} \\ \ddot{Y} \end{pmatrix}$. On détermine d'abord $\dot{A}_t$. Comme $\alpha = \omega t$ (en prenant $\alpha(t = 0) = 0$), on obtient immédiatement que $\dot{A}_t = \begin{pmatrix} -\omega \sin(\omega t) & -\omega \cos(\omega t) \\ \omega \cos(\omega t) & -\omega \sin(\omega t) \end{pmatrix} = -\omega J A_t$ où

$J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$. On en déduit que :

$$\begin{pmatrix} \ddot{X} \\ \ddot{Y} \end{pmatrix} = A_t \begin{pmatrix} \ddot{x} - 2\omega\dot{y} - \omega^2x \\ \ddot{y} + 2\omega\dot{x} - \omega^2y \end{pmatrix} \quad (3)$$

Avec l'équation 1 multipliée par A_t^{-1} on a $m A_t^{-1} \begin{pmatrix} \ddot{X} \\ \ddot{Y} \end{pmatrix} = -A_t^{-1} \begin{pmatrix} \mathcal{U}_X \\ \mathcal{U}_Y \end{pmatrix}$. Le membre de gauche se simplifie en $m \times$ le membre de gauche de l'équation 3. Le membre de droite correspond cependant au changement de base de la dérivée de l'énergie potentielle : le membre de droite devient $-\begin{pmatrix} \mathcal{U}_x \\ \mathcal{U}_y \end{pmatrix}$. En posant $\bar{\mathcal{U}}_x := -\mathcal{U}_x + m\omega^2x$ et $\bar{\mathcal{U}}_y := -\mathcal{U}_y + m\omega^2y$, on obtient finalement les équations du mouvement suivantes :

$$\begin{cases} \ddot{x} - 2\omega\dot{y} = -\frac{1}{m}\bar{\mathcal{U}}_x \\ \ddot{y} + 2\omega\dot{x} = -\frac{1}{m}\bar{\mathcal{U}}_y \end{cases} \quad (4)$$

ainsi que l'équation de l'énergie potentielle effective dans R_{NG} :

$$\bar{U} = U(x, y) - \frac{1}{2}m\omega^2(x^2 + y^2) \quad (5)$$

On peut tracer ce potentiel à l'aide d'un programme python disponible dans la section python. On obtient alors la figure suivante :

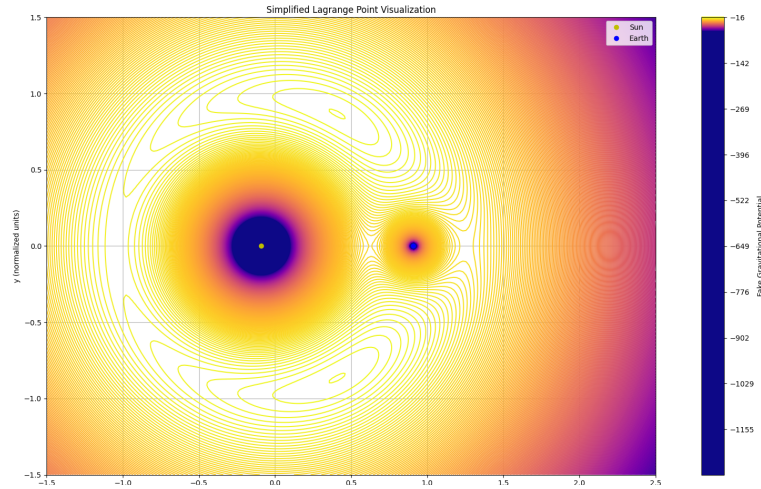


FIGURE 2 – A nice potential.

1.4 Coordonnées des points colinéaires

On cherche dans cette section à déterminer les coordonnées des points colinéaires, i.e. ceux que l'on peut observer sur l'axe des abscisses dans la figure (2). Ainsi, dans cette partie, nous travaillerons en supposant aucun mouvement vertical : $\forall t, y(t) = 0, v_y(t) = 0$ and $a_y(t) = 0$. Les points d'équilibres colinéaires correspondent ici aux points d'annulations de la dérivée de $\bar{U}(x, 0)$ par

rapport à x . Notons $\mu := \frac{M_2}{M_1 + M_2}$ le coefficient de masse du système. Ainsi, le barycentre du système étant placé en $(0, 0)$, on a :

$$\bar{U}(x, 0) = -\frac{1}{2}m\omega^2 x^2 - \frac{GmM_1}{|x + \mu R|} - \frac{GmM_2}{|x - (1 - \mu)R|} \quad (6)$$

et en dérivant :

$$\frac{\partial \bar{U}}{\partial x} = -m\omega^2 x + \frac{GmM_1}{(x + \mu R)^2} \text{sign}(x + \mu R) + \frac{GmM_2}{(x - (1 - \mu)R)^2} \text{sign}(x - (1 - \mu)R) \quad (7)$$

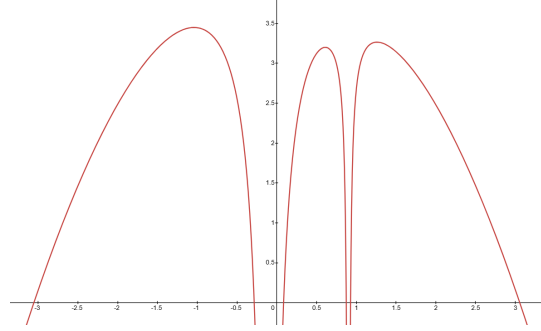


FIGURE 3 – Le potentiel effectif selon x en $y = 0$.

En traçant ce potentiel, on obtient alors la figure 3 précédente, sur laquelle on peut en observer la présence des trois points d'équilibre que l'on recherche. On les nommera $L3$, $L1$ et $L2$ selon leur ordre d'apparition sur la figure 3. On cherche maintenant à déterminer les points d'annulation du potentiel effectif (i.e. les points d'équilibres précédemment mentionnés). Pour ce faire, nous travaillerons avec l'hypothèse que $\mu \rightarrow 0^+$ qui nous simplifiera la résolution des calculs. Il peut être montré (pas discuté ici) que le polynôme de degré cinq, qui apparaît lors de la résolution analytique des points d'annulation de la formule 7, n'admet pas de racine calculable (cf. The Lagrange Points by Neil J. Cornish). On distingue trois cas.

1.4.1 Coordonnées de L1

On cherche ici à déterminer les coordonnées du point L1. Ce point se situe entre M_1 et M_2 . On prend alors $x \in]-\mu, 1 - \mu[$. Sur cet intervalle, on a $x + \mu R > 0$ et $x - (1 - \mu)R < 0$. En utilisant la forme dérivée de l'équation 6 et en se plaçant sur le segment considéré, la condition nécessaire d'équilibre s'écrit :

$$\omega^2 x - \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0 \quad (8)$$

On peut noter que les masses se simplifient. Du fait que l'on suppose $\mu \rightarrow 0$, on peut affirmer qualitativement que le point d'équilibre est bien plus proche de M_2 que de M_1 , sous peine que l'attraction de M_2 soit négligeable devant celle de M_1 . On pose alors $\varepsilon := x - (1 - \mu)R$ la distance entre M_2 et L1. Ici, on s'attend à ce que $\varepsilon < 0$ dû à l'orientation du référentiel. On obtient ainsi :

$$\omega^2((1 - \mu)R + \varepsilon) - \frac{GM_1}{(R + \varepsilon)^2} + \frac{GM_2}{\varepsilon^2} = 0 \quad (9)$$

On peut alors procéder à un DL à l'ordre 1 en ε (car on a émis l'hypothèse $\varepsilon \rightarrow 1$) ce qui, après quelques calculs longs donne :

$$\varepsilon = -\left(\frac{\mu}{3}\right)^{\frac{1}{3}}R \quad (10)$$

1.4.2 Coordonnées de L2

Déterminer les coordonnées de L2 est très similaire au processus pour L1. Dans ce cas, le point se trouve à l'opposé de M_2 par rapport à M_1 . Ainsi on a $x \in]1 - \mu, +\infty[$. Sur cet intervalle, on a $x + \mu R > 0$ et $x - (1 - \mu)R > 0$. On peut ainsi écrire comme précédemment l'équation liée à la condition nécessaire d'équilibre. Pour L2, nous pouvons reposer le même ε et tenir le même raisonnement pour obtenir au final que

$$\varepsilon = \left(\frac{\mu}{3}\right)^{\frac{1}{3}}R \quad (11)$$

Dans ce cas, on peut raisonner à nouveau qualitativement pour justifier ce ε . Le point L2 se situe au delà de l'astre M_2 ce qui implique que l'attraction de M_2 et celle de M_1 attire dans le sens $M \rightarrow M_2$ (où M est le point matériel associé au satellite, défini ainsi au 1.1). Or, si l'on se place loin de M_2 , seul l'attraction de M_1 sera ressentie, ce qui revient à traiter le problème à 2-corps, et on peut facilement montrer que l'on obtient une orbite pour laquelle la coordonnée en y dans R_{NG} est non constante égale à 0. Donc on n'aurait pas à faire à un point d'équilibre. Il faut ainsi se placer proche de M_2 .

1.4.3 Coordonnées de L3

Dernier des points colinéaires, L3 est bien différent de ses deux frères L1 et L2. Ce point se positionne de l'autre côté de M_1 par rapport à M_2 , et se situe, comme l'on va le voir, environ à la même distance de M_1 que M_2 . On a alors $x \in]-\infty, -\mu[$. Sur cet intervalle, on se retrouve avec $x + \mu R < 0$ et $x - (1 - \mu)R < 0$, ce qui se traduit dans l'équation liée à la condition nécessaire d'équilibre par :

$$\omega^2 x + \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0 \quad (12)$$

Or, avec l'hypothèse que $\mu \rightarrow 0$, qui se traduit par $M_2 \ll M_1$, on obtient l'approximation $x + \frac{GM_1}{(x + \mu R)^2} = 0$ d'où $x \approx -R$. On cherche maintenant à déterminer l'écart que l'on note δ entre l'approximation et la véritable position d'équilibre. On a alors $x = \delta - R$. En remplaçant dans 12 on a :

$$\omega^2(\delta - R) + \frac{GM_1}{(\delta - (1 - \mu)R)^2} + \frac{GM_2}{(\delta - (2 - \mu)R)^2} = 0 \quad (13)$$

On peut maintenant procéder à résoudre pour δ à l'aide de deux développements limités, un sur chacun des deux termes droits (DL en $\frac{\delta}{R} + \mu$). On peut ainsi montrer que :

$$\delta = \frac{\left(\frac{3}{4} - \frac{\mu}{4}\right)M_2 - 2\mu M_1}{3M_1 + \frac{5}{4}M_2} \quad (14)$$

1.5 Coordonnées des points équilatéraux

Comme spoilé par le nom de cette section, les points L4 et L5 se trouvent aux sommets des deux triangles équilatéraux dont les deux autres sommets sont M_1 et M_2 . Dans cette partie, nous nous

plaçons dans le plan $(O, \vec{u}_x, \vec{u}_y)$, comme défini au 1.3. On définit r_1 (*resp.* r_2) comme la distance entre M_1 (*resp.* M_2) et la position (x, y) du satellite. On peut alors exprimer $\bar{U}$ en fonction de r_1, r_2, x, y ainsi que r_1 et r_2 en fonction de x et y :

$$\begin{cases} r_1 = \sqrt{(x + \mu R)^2 + y^2} \\ r_2 = \sqrt{(x - (1 - \mu)R)^2 + y^2} \\ \bar{U}(x, y) = -\frac{1}{2}m\omega^2(x^2 + y^2) - \frac{GmM_1}{r_1} - \frac{GmM_2}{r_2} \end{cases} \quad (15)$$

ce qui donne en dérivant $\bar{U}$ selon x , et selon y :

$$\begin{cases} \frac{\partial \bar{U}}{\partial x} = -m\omega^2 x + \frac{GmM_1(x + \mu R)}{r_1^3} + \frac{GmM_2(x - (1 - \mu)R)}{r_2^3} \\ \frac{\partial \bar{U}}{\partial y} = -m\omega^2 y + \frac{GmM_1 y}{r_1^3} + \frac{GmM_2 y}{r_2^3} \end{cases} \quad (16)$$

On peut alors remplacer $\omega^2 = \frac{G(M_1 + M_2)}{R^3}$ dans les expressions et retrouver ainsi, en imposant la condition nécessaire d'équilibre, et après simplification, que :

$$\begin{cases} \frac{x}{R^3} = \frac{(1 - \mu)(x + \mu R)}{r_1^3} + \frac{\mu(x - (1 - \mu)R)}{r_2^3} = \left(\frac{1 - \mu}{r_1^3} + \frac{\mu}{r_2^3}\right)x + \frac{\mu(1 - \mu)R}{r_1^3} + \frac{\mu(1 - \mu)R}{r_2^3} \\ \frac{1}{R^3} = \frac{1 - \mu}{r_1^3} + \frac{\mu}{r_2^3} \end{cases} \quad (17)$$

et en remplaçant la deuxième expression dans la première, on obtient après simplification que :

$$r_1 = r_2 \quad \text{et} \quad \frac{1}{R^3} = \frac{1}{r_1^3} \iff r_1 = R \quad (18)$$

ce qui démontre que L_i , $i \in \{4, 5\}$ est un sommet d'un triangle équilatéral dont les deux autres sommets sont M_1 et M_2 . Notre étude étant menée dans le plan $(O, \vec{u}_x, \vec{u}_y)$, il n'existe que deux triangles équilatéraux qui vérifient cela. On décide arbitrairement de nommer L4 le point d'équilibre en ordonnées positives, et donc L5 celui en ordonnées négatives (cf figure ??).

NB : On peut remarquer que L4 est alors en avance de 60° par rapport à l'orbite de M_1 , et que L5 est en retard de phase identique. Cela permet de les caractériser indépendamment du référentiel choisi.

1.6 Représentation des points de Lagrange

Grâce aux sections précédentes, on peut récapituler les coordonnées de chacun des points de Lagrange. On a en effet les valeurs approchées pour les points colinéaires, et un peu de trigonométrie simple nous donne immédiatement les coordonnées des points équilatéraux.

Point	Coordonnée en x	Coordonnée en y
L1	$((1 - \mu) - (\frac{M_2}{3M_1 + M_2})^{\frac{1}{3}})R$	0
L2	$((1 - \mu) + (\frac{M_2}{3M_1 + M_2})^{\frac{1}{3}})R$	0
L3	$(-1 + \frac{(\frac{3}{4} - \frac{\mu}{4})M_2 - 2\mu M_1}{3M_1 + \frac{5}{4}M_2})R$	0
L4	$((\frac{1}{2} - \mu)R)$	$\frac{\sqrt{3}}{2}R$
L5	$((\frac{1}{2} - \mu)R)$	$-\frac{\sqrt{3}}{2}R$

TABLE 1 – Coordonnées des cinq points de Lagrange

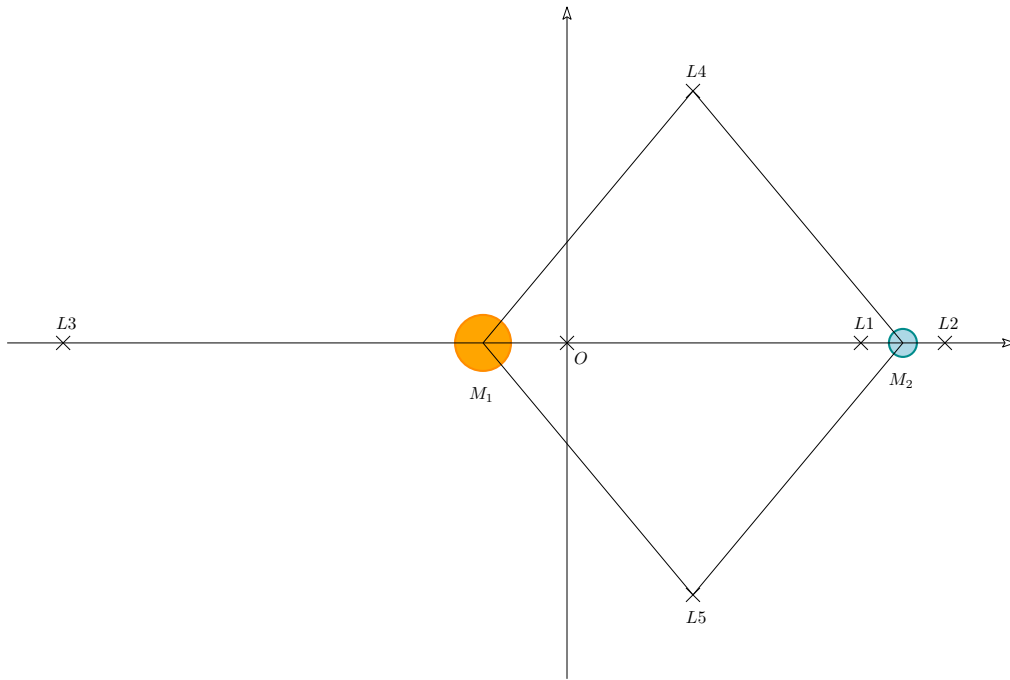


FIGURE 4 – Représentation de R_{NG} avec les points d'équilibre marqués.

2 Stabilité des Points de Lagrange

Nous avons établis que le PCR3BP admet cinq points d'équilibres. Il n'est pas compliqué de montrer que cela en va aussi pour le CR3BP (c'est-à-dire les cas où $z = 0$ n'est pas nécessairement vérifiée) et que ce sont les mêmes points d'équilibres (on se ramène en fait au PCR3BP simplement). Il suit que nous devons étudier la stabilité de ces points. La figure (2) nous permet d'émettre des conjectures, mais pour une étude rigoureuse, il faut linéariser les équations du mouvement autour de chaque point, ce qui nous permettra de conclure cas par cas sur la stabilité des points.

2.1 Linéarisation des équations du mouvement

On se place en (x_i, y_i) la position du $i^{\text{ème}}$ point de Lagrange. Avec la relation (4) on obtient que

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{v}_x = \ddot{x} = 2\omega\dot{y} - \frac{1}{m}\bar{\mathcal{U}}_x \\ \dot{v}_y = \ddot{y} = -2\omega\dot{x} - \frac{1}{m}\bar{\mathcal{U}}_y \end{cases} \quad (19)$$

i.e. il existe $f : \mathbb{R}^4 \rightarrow \mathbb{R}^4$ tel que $\dot{X} = f(X)$ où $X = (x, y, v_x, v_y)^T$ est l'état du système. On cherche alors à linéariser (19) de manière à obtenir $A \in \mathcal{M}_{4,4}(\mathbb{R})$ qui vérifie autour de l'équilibre que $\dot{X} = AX$. On cherche alors à linéariser $\bar{\mathcal{U}}$. Grâce à l'expansion en série de Taylor à l'ordre 1 en (x_i, y_i) on obtient

$$\begin{cases} \bar{\mathcal{U}}_x(x, y) = \bar{\mathcal{U}}_x(x_i, y_i) + \frac{\partial \bar{\mathcal{U}}_x}{\partial x}(x_i, y_i) \times (x - x_i) + \frac{\partial \bar{\mathcal{U}}_x}{\partial y}(x_i, y_i) \times (y - y_i) \\ \bar{\mathcal{U}}_y(x, y) = \bar{\mathcal{U}}_y(x_i, y_i) + \frac{\partial \bar{\mathcal{U}}_y}{\partial x}(x_i, y_i) \times (x - x_i) + \frac{\partial \bar{\mathcal{U}}_y}{\partial y}(x_i, y_i) \times (y - y_i) \end{cases} \quad (20)$$

que l'on peut plus simplement écrire, en reprenant les mêmes notations que précédemment, et en considérant le point L_i comme l'origine (donc que $(x_i, y_i) = (0, 0)$) comme

$$\begin{cases} \bar{\mathcal{U}}_x(x, y) = \bar{\mathcal{U}}_x^i + \bar{\mathcal{U}}_{xx}^i x + \bar{\mathcal{U}}_{xy}^i y \\ \bar{\mathcal{U}}_y(x, y) = \bar{\mathcal{U}}_y^i + \bar{\mathcal{U}}_{yx}^i x + \bar{\mathcal{U}}_{yy}^i y \end{cases} \quad (21)$$

On peut alors linéariser les équations du mouvement et on obtient

$$\dot{X} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{v}_x \\ \dot{v}_y \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{1}{m}\bar{\mathcal{U}}_{xx}^i & -\frac{1}{m}\bar{\mathcal{U}}_{xy}^i & 0 & 2\omega \\ -\frac{1}{m}\bar{\mathcal{U}}_{yx}^i & -\frac{1}{m}\bar{\mathcal{U}}_{yy}^i & -2\omega & 0 \end{bmatrix}}_{=:A} \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix} \quad (22)$$

cependant le reste du calcul dépendra du point étudié. On va donc étudier la stabilité de L1 et L2 ensemble, puis celle de L3, et pour finir celle de L4 et L5.

2.2 Stabilité de L1 et L2

2.2.1 Calcul explicite approché du linéarisé

Dans cette situation, on a exactement le potentiel de la relation (6). On peut alors dériver la relation (7) selon x et selon y . Pour simplifier le processus, on va d'abord dériver un élément $V := -\frac{a}{r(X,y)}$ où $r(X,y) = \sqrt{X^2 + y^2}$. On a

$$\begin{cases} \mathcal{V}_x = \frac{aX}{r^3} \\ \mathcal{V}_y = \frac{ay}{r^3} \end{cases} \implies \begin{cases} \mathcal{V}_{xx} = \frac{a}{r^3} + X \frac{-3aX}{r^5} = \frac{r^2 - 3X^2}{r^5} a \\ \mathcal{V}_{xy} = aX \frac{\partial}{\partial y} \left(\frac{1}{r^3} \right) = -\frac{3aXy}{r^5} \\ \mathcal{V}_{yx} = ay \frac{\partial}{\partial x} \left(\frac{1}{r^3} \right) = -\frac{3aXy}{r^5} \\ \mathcal{V}_{yy} = \frac{a}{r^3} + y \frac{-3ay}{r^5} = \frac{r^2 - 3y^2}{r^5} a \end{cases} \quad (23)$$

et utilisant ces formules avec les variables du problème on obtient

$$\begin{cases} \bar{\mathcal{U}}_{xx} = -m\omega^2 + GmM_1 \frac{r_1^2 - 3(x+\mu R)^2}{r_1^5} + GmM_2 \frac{r_2^2 - 3(x-(1-\mu)R)^2}{r_2^5} \\ \bar{\mathcal{U}}_{xy} = \bar{\mathcal{U}}_{yx} = -3Gm \left(\frac{M_1(x+\mu R)y}{r_1^5} + \frac{M_2(x-(1-\mu)R)y}{r_2^5} \right) \\ \bar{\mathcal{U}}_{yy} = -m\omega^2 + GmM_1 \frac{r_1^2 - 3y^2}{r_1^5} + GmM_2 \frac{r_2^2 - 3y^2}{r_2^5} \end{cases} \quad (24)$$

On peut alors remplacer les coordonnées de L1 dans ces équations (on fait alors une composition de développements limités). On sait que L1 a pour coordonnée $((1-\mu)R - (\frac{\mu}{3})^{\frac{1}{3}}R, 0)$ ce qui nous permet d'écrire $r_1^2(L_1) = (1 - (\frac{\mu}{3})^{\frac{1}{3}}R)^2$ ainsi que $r_2^2(L_1) = (\frac{\mu}{3})^{\frac{2}{3}}R^2$. On peut reconnaître, grâce à $y = 0$, que

$$\begin{cases} \bar{\mathcal{U}}_{xx} = -m\omega^2 - 2Gm \left(\frac{M_1}{r_1^3} + \frac{M_2}{r_2^3} \right) \\ \bar{\mathcal{U}}_{yy} = -m\omega^2 + Gm \left(\frac{M_1}{r_1^3} + \frac{M_2}{r_2^3} \right) \\ \bar{\mathcal{U}}_{xy} = \bar{\mathcal{U}}_{yx} = 0 \end{cases} \quad (25)$$

et on observe que $\frac{M_2}{r_2^3} = \frac{M_2}{\frac{\mu}{3}R^3} = 3\omega^2$ et que $\frac{M_1}{r_1^3} = \frac{M_1}{(R - (\frac{\mu}{3})^{\frac{1}{3}}R)^3} \sim \frac{3M_1}{R^3} \sim \omega^2$ en supposant que $\mu \rightarrow 0$ et que $r_1 \simeq R$ (ce qui est vraie si la supposition précédente est correcte). On obtient ainsi, en remplaçant dans les équations précédentes, que

$$\begin{cases} \bar{\mathcal{U}}_{xx} = -9m\omega^2 \\ \bar{\mathcal{U}}_{yy} = 3m\omega^2 \\ \bar{\mathcal{U}}_{xy} = \bar{\mathcal{U}}_{yx} = 0 \end{cases} \quad (26)$$

Le processus est le même pour le point L2. Il faut noter que une partie de la littérature trouvée en ligne affirme que $\frac{1}{m}\bar{\mathcal{U}}_{xx} = \mp\omega^2$. Cette erreur apparaît plusieurs fois à travers plusieurs papiers différents, cependant on peut remarquer la présence dans chacun de ces papiers une référence à un texte éducatif de Neil J. Cornish appelé *The Lagrange Points* dans lequel il affirme cette égalité. Nous continuerons l'étude en supposant cette littérature erronée.

La linéarisation des équations du mouvement devient alors pour L1 et L2

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 9\omega^2 & 0 & 0 & 2\omega \\ 0 & -3\omega^2 & -2\omega & 0 \end{bmatrix} \quad (27)$$

et on peut maintenant chercher les valeurs propres de cette matrice, de manière à obtenir une solution de l'équation différentielle $\dot{X} = AX$, ce qui nous permettra d'obtenir une compréhension et de la stabilité du système, et de l'espace de phase.

2.2.2 Réduction du linéarisé sur les sous-espaces stables réels

On calcule le polynôme caractéristique de A :

$$\begin{aligned}\chi_A &= \begin{vmatrix} X & 0 & -1 & 0 \\ 0 & X & 0 & -1 \\ -9\omega^2 & 0 & X & -2\omega \\ 0 & 3\omega^2 & 2\omega & X \end{vmatrix} = X \begin{vmatrix} X & 0 & -1 \\ 0 & X & -2\omega \\ 3\omega^2 & 2\omega & X \end{vmatrix} - \begin{vmatrix} 0 & X & -1 \\ -9\omega^2 & 0 & -2\omega \\ 0 & 3\omega^2 & X \end{vmatrix} \\ &= X^2 \begin{vmatrix} X & -2\omega \\ 2\omega & X \end{vmatrix} - X \begin{vmatrix} 0 & X \\ 3\omega^2 & 2\omega \end{vmatrix} - 9\omega^2 \begin{vmatrix} X & -1 \\ 3\omega^2 & X \end{vmatrix} = X^2(X^2 + 4\omega^2) + 3\omega^2 X^2 - 9\omega^2(X^2 + 3\omega^2) \\ &= X^4 - 2\omega^2 X^2 - 27\omega^2\end{aligned}\tag{28}$$

qui est un polynôme de degré deux en X^2 dont les racines sont

$$\Delta = 112\omega^2 > 0 \implies \begin{cases} r_1 = \omega^2(1 + 2\sqrt{7}) > 0 \\ r_2 = \omega^2(1 - 2\sqrt{7}) < 0 \end{cases}\tag{29}$$

ce qui nous permet de factoriser χ_A , et en utilisant l'identité remarquable $a^2 - b^2 = (a + b)(a - b)$ on peut factoriser encore une fois, en remarquant que une solution de $z^2 = r_2$, d'inconnue $z \in \mathbb{C}$ est $i\omega\sqrt{2\sqrt{7} - 1}$. On obtient alors la forme factorisée de χ_A , à savoir

$$\chi_A = (X - \lambda)(X + \lambda)(X - i\nu)(X + i\nu) \text{ où } \begin{cases} \lambda := \omega\sqrt{1 + 2\sqrt{7}} \in \mathbb{R}_+ \\ \nu := \omega\sqrt{2\sqrt{7} - 1} \in \mathbb{R}_+ \end{cases}\tag{30}$$

ce qui, en notant $Z(P)$ les racines de $P \in \mathbb{R}[X]$, s'écrit simplement $Z(\chi_A) = \{\lambda, -\lambda, i\nu, -i\nu\}$. Et comme on est en dimension finie (à savoir 4 ici), les racines de χ_A sont exactement les valeurs propres de A : $Sp(A) = Z(\chi_A)$.

Soit $z \in \mathbb{C} \setminus \{0\}$ tel que $Az = i\nu z$. Il existe alors $(x, y) \in \mathbb{R}^2$ tel que $z = x + iy$, ce qui donne $Az = Ax + iAy = i\nu x - \nu y$ car $A \in \mathcal{M}_n(\mathbb{R})$. Par unicité de l'écriture complexe, on a que $Ax = -\nu y$ et $Ay = \nu x$ ce qui caractérise l'action de la matrice sur le plan réel $Vect(x, y)$. En notant alors $B_d := (\eta, \xi, \zeta_1, \zeta_2)$ une base où les deux premiers vecteurs sont propres pour λ et $-\lambda$, et où les ζ_i sont tel que $\zeta_1 + i\zeta_2$ est un vecteur propre pour la valeur propre $i\nu$. Avec ces notations, A s'écrit alors

$$A = P \begin{bmatrix} \lambda & 0 & 0 & 0 \\ 0 & -\lambda & 0 & 0 \\ 0 & 0 & 0 & \nu \\ 0 & 0 & -\nu & 0 \end{bmatrix} P^{-1}\tag{31}$$

avec P la matrice de passage entre la base originale et B_d .

L'intérêt de cette écriture réside alors dans l'explicitation d'une action d'oscillateur harmonique sur un plan stable de l'espace de phase. Cela nous permet aussi d'éviter la présence de nombres complexes, facilitant notre raisonnement.

2.2.3 Calcul de la matrice de passage

Dans cette partie, on cherche à calculer les vecteurs propres η , ξ , et $\zeta_1 + i\zeta_2 =: \zeta$ associés respectivement aux valeurs propres λ , $-\lambda$ et $i\nu$. On résout alors les équations

$$\begin{cases} A\eta = \lambda\eta \\ A\xi = -\lambda\xi \\ A\zeta = i\nu\zeta \end{cases} \quad (32)$$

que l'on simplifie en l'équation (E) : $AX = \rho X$, d'inconnue $X \in \mathcal{M}_{4,1}(\mathbb{C})$, où ρ désigne une valeur propre de A .

Pour ce faire, on pose $X =: (x_1, x_2, x_3, x_4)$. Alors X vérifie l'équation (E) si, et seulement si X vérifie le système

$$\begin{cases} x_3 = \rho x_1 \\ x_4 = \rho x_2 \\ 9\omega^2 x_1 + 2\omega x_4 = \rho x_3 \\ -3\omega^2 x_2 - 2\omega x_3 = \rho x_4 \end{cases} \iff \begin{cases} x_3 = \rho x_1 \\ x_4 = \rho x_2 \\ (9\omega^2 - \rho^2)x_1 + 2\omega\rho x_2 = 0 \\ (-3\omega^2 - \rho^2)x_2 - 2\omega\rho x_1 = 0 \end{cases} \quad (33)$$

On résout alors le système de Cramer 2×2 défini par les lignes 3 et 4. Or, si le déterminant de ce système est non-nul, alors $x_1 = x_2 = 0$, et comme on cherche des solutions non-nulles, on se place dans le cas où les lignes du système sont liées. Avec la ligne 3 par exemple, on obtient

$$x_2 = \frac{\rho^2 - 9\omega^2}{2\omega\rho} x_1 \iff X = \begin{bmatrix} 1 \\ \frac{\rho^2 - 9\omega^2}{2\omega\rho} \\ \rho \\ \frac{(\rho^2 - 9\omega^2)\rho}{2\omega\rho} \end{bmatrix} = \begin{bmatrix} 1 \\ \sigma \\ \rho \\ \sigma\rho \end{bmatrix}$$

Ainsi, la matrice de passage de la relation 31 s'écrit

$$P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -\sigma & \sigma & -i\tau & i\tau \\ \lambda & -\lambda & i\nu & -i\nu \\ -\lambda\sigma & -\lambda\sigma & \nu\tau & \nu\tau \end{bmatrix}, \text{ avec } \sigma = \frac{9\omega^2 - \lambda^2}{2\omega\lambda} \text{ et } \tau = -\frac{9\omega^2 + \nu^2}{2\omega\nu} \quad (34)$$

et on peut maintenant chercher la forme explicite des solutions de l'équation du mouvement linéarisé.

2.2.4 Calcul des solutions du linéarisé

On peut réécrire la relation (22) à savoir $\dot{X} = AX$ sous la forme $\dot{Y} = DY$ où $Y := P^{-1}X$ et D est la matrice dans la relation (31). Alors l'ensemble des solutions est

$$\mathcal{S}_{dz} = \{t \mapsto \begin{bmatrix} C_1 e^{\lambda t} \\ C_2 e^{-\lambda t} \\ C_3 \cos(\nu t) + C_4 \sin(\nu t) \\ -C_3 \sin(\nu t) + C_4 \cos(\nu t) \end{bmatrix}, (C_1, C_2, C_3, C_4) \in \mathbb{R}^4\} \quad (35)$$

et pour obtenir l'ensemble des solutions $\mathcal{S}$ de la relation (22) il suffit de calculer $X = PY$. De plus, comme on recherche les trajectoires, on peut se limiter au calcul des deux premières composantes de X , et après calcul on obtient que

$$X_{pos} \in \left\{ \begin{bmatrix} 2\omega\lambda(C_1e^{\lambda t} - C_2e^{-\lambda t}) + 2\omega\nu(-C_3\sin(\nu t) + C_4\cos(\nu t)) \\ (\lambda^2 - 9\omega^2)(C_1e^{\lambda t} + C_2e^{-\lambda t}) - (\nu^2 + 9\omega^2)(C_3\cos(\nu t) + C_4\sin(\nu t)) \end{bmatrix}, (C_1, C_2, C_3, C_4) \in \mathbb{R}^4 \right\} \quad (36)$$

On observe clairement que la position présente un comportement oscillatoire de fréquence $\frac{\nu}{2\pi}$ modulé par une exponentielle, dont le temps caractéristique est $\frac{1}{\lambda}$. On peut alors faire une application numérique avec l'expression approché de λ de la relation (30), ce qui donne dans le cadre du système Terre-Lune, puis du système Terre-Soleil

$$\begin{cases} \frac{1}{\lambda_{Lune/Terre}} = \frac{T_{Lune}}{2\pi} \frac{1}{\sqrt{1+2\sqrt{7}}} \cong \frac{28 \times 24 \times 3600}{2\pi} \times \frac{1}{\sqrt{1+2\sqrt{7}}} = 1,54.10^5 s = 1,8 \text{ jours} \\ \frac{1}{\lambda_{Terre/Soleil}} = 2,0.10^6 s = 23 \text{ jours} \end{cases} \quad (37)$$

Cependant, ces valeurs sont à prendre avec des pincettes du aux approximations faites. Pour plus de détails, on peut utiliser des méthodes de résolutions numériques qui permettent de calculer précisément les valeurs propres, ainsi que les constantes σ et τ . Dans le cadre du système Terre-Lune, on obtient grâce à une évaluation numérique de la matrice du linéarisé, et du module **numpy**, on obtient que

$$\begin{cases} \lambda = 7.81469 \cdot 10^{-6} s^{-1} \\ \nu = 6,62217 \cdot 10^{-6} s^{-1} \\ \sigma = 3,39222 \\ \tau = -3,58654 \end{cases} \quad (38)$$

Ces valeurs nous permettront de faire de meilleures prédictions pour les conditions initiales d'orbites dans le système non-linéaire, ainsi qu'avoir une meilleure précision quand à la période de ces dernières.

3 Approche asymptotique des orbites autour de L1 et L2

On a établi dans la partie précédente que les points L1 et L2 sont des points "cols". La forme des solutions données par la relation 36 montre l'existence d'orbites dans le système linéarisé (lorsque les conditions initiales forcent $C_1 = C_2 = 0$). On remarque que dans le cas où seule l'une des deux constantes C_1 ou C_2 est nulle, on approche (avec t croissant ou décroissant) asymptotiquement la solution périodique du linéarisé. Cette partie cherche à étudier ces directions dans l'espace de phase qui permet de tendre asymptotiquement vers une solution périodiques. Ces directions sont les "variétés", dont l'une (celle associée à $-\lambda$) est dite "stable" et l'autre (celle associée à λ) est dite "instable". Le calcul de ces dernières permet la fabrication de trajectoires coutant moins chères en carburant.

3.1 Existence d'orbites périodiques

Comme dit précédement, les solutions du système linéarisé se mettent sous la forme donnée par la relation 36. Ainsi le système admet les solutions particulières suivantes :

$$\mathcal{S}_{periodiques} = \left\{ \begin{bmatrix} \alpha e^{i\nu t} + \beta e^{-i\nu t} \\ -i\alpha\tau e^{i\nu t} + i\tau\beta e^{-i\nu t} \end{bmatrix}, (\alpha, \beta) \in \mathbb{R}^2 \right\} \quad (39)$$

qui sont clairement périodique, de période $\frac{2\pi}{\nu}$.

3.2 Introduction aux variétés d'une orbite périodique

Définition. Soit un système différentiel $\dot{X} = F(X)$ où $F \in \mathcal{C}^1(\mathbb{R}^n, \mathbb{R}^n)$. Pour une condition initiale x_0 , on pose $\varphi(t, x_0)$ l'état du système à un instant t tel que $\varphi(t_0, x_0) = x_0$. Soit $\gamma \in \mathcal{F}(\mathbb{R}, \mathbb{R}^{2n})$ une orbite de référence de période T . Alors on définit la variété *stable* (resp. *instable*) associée à γ notée $W^s(\Gamma)$ (resp. $W^u(\Gamma)$) comme

$$W^s(\Gamma) = \{x_0 \in \mathbb{R}^n | d(\varphi(t, x_0), \Gamma) \xrightarrow[t \rightarrow +\infty]{} 0\}$$

(resp. *quand* $t \rightarrow -\infty$), où $\Gamma = \{\gamma(t), t \in \mathbb{R}\}$.

On cherche à calculer alors les variétés stables et instables associées à une orbite donnée. On déterminera d'abord l'approximation numérique des variétés à l'aide de la matrice de monodromie. A partir de cela, on intégrera le vecteur d'état associé à la variété étudiée selon les équations non-linéaires.

3.3 La matrice de Monodromie

On se fixe $\gamma \in \mathcal{S}_{periodiques}$ une solution T -périodique du système différentiel défini par la relation 4. Pour tout $t \in \mathbb{R}$, on note $A(t)$ le linéarisé de ce système en le point de l'espace des phases $\gamma(t)$. Par périodicité de γ , A est aussi T -périodique. Alors on étudie le système différentiel $\dot{X} = AX$ d'inconnue $X \in \mathcal{C}^1(\mathbb{R}, \mathbb{R}^4)$, à coefficients T -périodiques.

Définition. Soit $(E) \dot{X} = AX$ un système différentiel. Un système fondamental de solutions de l'équation différentielle (E) est une famille $(y_1, \dots, y_n)$ de n solutions indépendantes de (E) . On appelle *matrice fondamentale du système* une matrice constituée de ces solutions sous forme de vecteurs colonnes (c'est un abus de notation, car c'est en toute rigueur une *application*, mais on réfèrera à l'application, et la matrice en un point t , de la même manière).

Définition. On appelle *matrice fondamentale principale du système* l'unique matrice fondamentale Φ qui vérifie que $\Phi(0) = I_n$.

Montrons que cette dernière définition est légitime. Soit Φ, Ψ deux matrices fondamentales. Montrons que $\exists M \in GL_n(\mathbb{K}), \Phi = \Psi M$ où M est indépendante du temps, et que réciproquement, si $M \in GL_n(\mathbb{K})$, alors ΨM est une matrice fondamentale du système. Soit $y_0 \in \mathbb{K}^n$. Par théorème de Cauchy-Lipschitz linéaire, il existe une unique $y \in \mathcal{S}$ vérifiant $y(t_0) = y_0$. Alors on a $\forall t \in \mathbb{R}, y(t) = \Phi(t)\Phi(t_0)^{-1}y_0$. De même, on a $\forall t \in \mathbb{R}, y(t) = \Psi(t)\Psi(t_0)^{-1}y_0$. Donc

$$\forall t \in \mathbb{R}, \Psi(t)\Psi(t_0)^{-1}y_0 = \Phi(t)\Phi(t_0)^{-1}y_0 \quad (40)$$

Et cette égalité étant vraie pour tout $y_0 \in \mathbb{K}^n$, on a

$$\forall t \in \mathbb{R}, \Psi(t)\Psi(t_0)^{-1} = \Phi(t)\Phi(t_0)^{-1}. \quad (41)$$

ce qui, en posant $M := \Psi(t_0)^{-1}\Phi(t_0)$, on obtient bien

$$\forall t \in \mathbb{R}, \Phi = \Psi M \quad (42)$$

avec M est inversible en tant que produit de matrices inversibles.

Théorème. de Floquet : Soit $X' = AX$ une équation différentielle où A est à coefficients T -périodiques, et Φ la matrice fondamentale principale du système. Alors il existe $P \in \mathcal{C}_T^1(\mathbb{R}, (M)_n(\mathbb{C}))$ et $B \in (M)_n(\mathbb{C})$ tel que

$$\forall t \in \mathbb{R}, \Phi(t) = P(t)e^{tB}$$

que l'on appelle la *forme normale de Floquet*. En particulier, on appelle $e^{TB} = \Phi(T)$ la *matrice de Monodromie* du système, et *multiplicateurs caractéristiques du système* les valeurs propres de la matrice de Monodromie. Les multiplicateurs caractéristiques sont non nuls et indépendants du choix de Φ et du choix de B .

Démonstration : Soit $t \in \mathbb{R}$. Par T -périodicité de A , on a

$$\Phi'(t+T) = A(t+T)\Phi(t+T) = A(t)\Phi(t+T) \quad (43)$$

De plus, $\det(\Phi(t+T)) = \det(\Phi(t)) \neq 0$ donc $t \mapsto \Phi(t+T)$ est une matrice fondamentale du système car vérifiant l'équation différentielle, et inversible.

Alors il existe $M \in \mathcal{M}_n(\mathbb{C})$ tel que $\Phi(t+T) = \Phi(t)M$, ($M = \Phi(T)$ en évaluant en 0). Par surjectivité de l'exponentielle sur $GL_n(\mathbb{C})$, il existe $B \in \mathcal{M}_n(\mathbb{C})$ tel que $M = e^{TB}$. On pose alors $\forall t \in \mathbb{R}, P(t) := \Phi(t)e^{-tB}$. Ainsi P est une application continue de $\mathbb{R}$ dans $GL_n(\mathbb{C})$, et pour tout $t \in \mathbb{R}$,

$$P(t+T) = \Phi(t+T)e^{-(t+T)B} = \Phi(t)Me^{-TB}e^{-tB} = \Phi(t)e^{-tB} = P(t) \quad (44)$$

car e^{-tB} et e^{-TB} commutent. Alors P est T -périodique.

On a ainsi $\Phi(T) = P(T)e^{TB} = P(0)e^{TB} = \Phi(0)e^{TB} = e^{TB}$ la matrice de Monodromie, car Φ est la matrice fondamentale principale. Comme elle est inversible, ses valeurs propres sont non-nulles. Soit Ψ une matrice fondamentale du même système, et $C \in \mathcal{M}_n(\mathbb{C})$ qui vérifie que $\Psi(0)^{-1}\Psi(T) = e^{TC}$. Il existe $Q \in GL_n(\mathbb{C})$ tel que $\Psi = \Phi Q$. Alors

$$e^{TC} = \Psi(0)^{-1}\Psi(T) = (\Phi(0)Q)^{-1}(\Phi(T)Q) = Q^{-1}\Phi(T)Q = Q^{-1}e^{TB}Q \quad (45)$$

i.e. les matrices sont semblables. Or les valeurs propres sont un invariant de similitude. Donc les valeurs propres sont indépendantes du choix de B et de la matrice fondamentale.

3.4 Lien avec le flot

Définition. Soient une équation différentielle $X' = AX$, et Φ la matrice fondamentale principale de ce système. On appelle *flot* l'application définie pour $t \in \mathbb{R}$ fixé par $\forall x_0 \in \mathbb{K}^n, \eta(t, x_0) := \Phi(t)x_0$. Le flot pour un certain t est alors l'application qui, à une condition initiale x_0 donnée, associe le vecteur $x(t)$ où x est l'unique solution au problème de Cauchy de condition initiale x_0 .

Avec les mêmes notations que la définition précédente, on observe que la différentielle du flot en un x_0 est $x \mapsto \eta(t, x_0)$ car le flot est une application linéaire. On a alors $J\eta(t, x_0) = \Phi(t)$ la Jacobienne du flot. Ainsi, on voit que $\Phi(t)$ rend compte de la variation de l'état à un instant t pour une variation des conditions initiales. Alors la variation de l'état après une pseudo-orbite selon la variation des conditions initiales est quantifiée par la matrice de Monodrome $\Phi(T) = M$. On peut remarquer que $\forall k \in \mathbb{N}^*, \Phi(kT) = M^k$ donc la matrice de Monodromie quantifie exactement les variations après k orbites.

3.5 Étude théorique de la matrice de Monodromie

Comme on a vu précédemment, la matrice de Monodromie nous permet de quantifier la sensibilité aux variations de l'orbite γ . On cherche alors à étudier les valeurs propres, et les directions propres associées, pour déterminer les directions stables et instables de l'orbite. Le calcul des valeurs propres différentes de 1 nous aidera au calcul des variétés stable et instable de γ . On travaille avec la matrice fondamentale principale, donc on a toujours $M = \Phi(T)$ et $\Phi(0) = I_n$, en considérant 0 comme l'origine temporelle pour nos conditions initiales (i.e. $t_0 = 0$).

3.5.1 Première multiplicité de 1 comme valeur propre

On sait que γ vérifie l'équation différentielle du système non-linéaire $X' = f(X)$ où f est la fonction qui donne le système (19). De plus,

$$\forall t \in \mathbb{R}, \frac{d}{dt}\gamma'(t) = \frac{d}{dt}f(\gamma(t)) = Jf(\gamma(t)) \cdot \gamma'(t) = A(t)\gamma'(t) \quad (46)$$

donc γ' vérifie l'équation différentielle linéarisée. Alors $\gamma'(t) = \Phi(t)\gamma'_0$ pour tout instant t . En particulier, pour $t = T$, on a

$$\gamma'_0 = \gamma'(T) = \Phi(T)\gamma'_0 = M\gamma'_0 \quad (47)$$

i.e. γ'_0 est un vecteur propre de M pour la valeur propre 1.

3.5.2 Deuxième multiplicité de 1 comme valeur propre

La deuxième multiplicité de 1 est spécifique à notre cadre. On sait que l'énergie est une constante pour une trajectoire donnée, c'est-à-dire que pour une solution y de notre équation différentielle linéarisée, $\forall t \in \mathbb{R}, E(y(t)) = E(y(0))$, où $E : \mathbb{R}^4 \rightarrow \mathbb{R}$. Dans la suite de cette section, on travaille à t fixé : notre variable est x_0 la condition initiale, et on utilise alors le flot. L'égalité précédente se traduit par

$$\forall x_0 \in \mathbb{R}^4, E(\eta(t, x_0)) = E(x_0) \quad (48)$$

où η est le flot. On prend maintenant la jacobienne de l'égalité. Soit $x_0 \in \mathbb{R}^4$,

$$JE(\eta(x_0, t)) \cdot J\eta(x_0, t) = JE(x_0) \quad (49)$$

et en évaluant le flot en γ_0 la condition initiale de l'orbite de référence, on a $\eta(T, \gamma_0) = \gamma_0$. Ainsi on obtient

$$JE(\gamma_0)M = JE(\gamma_0) \quad (50)$$

ce qui montre bien que $JE(\gamma_0)$ est un vecteur propre pour la valeur propre 1.

Montrons que $JE(\gamma_0)$ et γ'_0 sont bien indépendants et donc que 1 est de multiplicité 2 ou plus. Pour cela, on utilise la conservation de l'énergie à nouveau, mais en travaillant avec t comme variable. On a $\forall t \in \mathbb{R}, \frac{dE}{dt}(\gamma(t)) = 0$. Or par formule de composition des dérivées, on a

$$\forall t \in \mathbb{R}, \frac{\partial E}{\partial x}(\gamma(t)) \cdot \frac{d\gamma}{dt}(t) = 0 \quad (51)$$

et en particulier

$$\frac{\partial E}{\partial x}(\gamma_0) \cdot \gamma'_0 = \left\langle \left(\frac{\partial E}{\partial x}(\gamma_0) \right)^T, \gamma'_0 \right\rangle = 0 \quad (52)$$

donc les deux vecteurs propres sont orthogonaux pour le produit scalaire usuel sur $\mathcal{M}_n(\mathbb{R})$. Ils sont donc indépendants.

3.6 Etude numérique de la matrice de Monodromie

Dans cette sous-partie, on cherche à expliciter une méthode pour calculer numériquement les directions propres d'une orbite d'amplitude fixée. Les caractéristiques de l'orbite dépendent directement de ce paramètre. La forme des solutions de l'équation linéarisée est

$$\mathcal{S}_{lin} = \left\{ t \rightarrow \begin{bmatrix} C_1 e^{\lambda t} + C_2 e^{-\lambda t} + R \cos(\nu t - \varphi) \\ -\sigma(C_1 e^{\lambda t} - C_2 e^{-\lambda t}) + \tau R \sin(\nu t - \varphi) \\ \lambda(C_1 e^{\lambda t} - C_2 e^{-\lambda t}) + \nu R \sin(\nu t - \varphi) \\ -\lambda\sigma(C_1 e^{\lambda t} + C_2 e^{-\lambda t}) + \nu\tau R \cos(\nu t - \varphi) \end{bmatrix}, (C_1, C_2, R, \varphi) \in \mathbb{R}^4 \right\} \quad (53)$$

alors pour une solution périodique, on a nécessairement $C_1 = C_2 = 0$. Cela nous laisse deux paramètres, à savoir l'amplitude et le déphasage de l'orbite. Ce dernier paramètre ne change pas la famille de courbes solutions obtenue, mais seulement la position de la condition initiale. On a alors une famille à un paramètre d'orbite périodique dans le système linéarisé.

3.6.1 Calcul d'une orbite

On se fixe une amplitude R_0 désirée selon l'axe Ox . On cherche une solution réelle ayant cette amplitude. Pour cela, on part de la solution que nous propose le système linéaire. Avec ce qui précède, une orbite du système linéaire d'amplitude R_0 selon l'axe Ox vérifie

$$\begin{cases} x(0) = R_0 \cos(\varphi) \\ y(0) = -\tau R_0 \sin(\varphi) \\ v_x(0) = -\nu R_0 \sin(\varphi) \\ v_y(0) = \nu \tau R_0 \cos(\varphi) \end{cases} \quad \text{ou s.p.d.g.} \quad \begin{cases} x(0) = R_0 \\ y(0) = 0 \\ v_x(0) = 0 \\ v_y(0) = \nu \tau R_0 \end{cases} \quad (54)$$

car quitte à déphaser l'orbite, on considère $\varphi = 0$ sans perte de généralité.

Pour avoir de meilleurs résultats, on se débarrasse de nos approximations pour λ et ν que l'on avait précédemment faite, et on utilise la fonction `linalg.eig` du module `numpy` qui renvoie les valeurs propres et les vecteurs propres d'une matrice donnée. Il faut alors se mouiller quand à nos données. Par la suite, nous prendrons $R_0 = 1000 \text{ km}$. Pour les valeurs propres du linéarisé en $L1$, ce n'est pas encore nécessaire. Un code python permet de calculer numériquement ces valeurs. On obtient alors

$$\begin{cases} \nu = 6,62217 \cdot 10^{-6} \text{ s}^{-1} \\ \tau = -3,58654 \end{cases} \quad (55)$$

et donc que $\nu \tau = 2,37 \cdot 10^{-5} \text{ s}^{-1}$. Alors on peut faire une prévision initiale $v_{y0} \approx 23,7 \text{ m.s}^{-1}$. Cette prévision va nous permettre d'initier une sorte de dichotomie qui cherchera à minimiser $v_x(t)$ à l'instant t qui vérifie que $y(t) = 0$ pour la première fois en dehors de $t = 0$. C'est ce qu'on va appeler *l'instant de premier retour* (à la droite $D : y = 0$ sous-entendu). La raison pour laquelle on cherche à faire cela est que si $v_x(t_{1^{er}\text{retour}}) = 0$, alors par symétrie du système, et son invariance selon le sens du temps, on aura une orbite périodique (Il y a toute une théorie qui utilise les applications de poincarés, où l'instant cherché en est un point fixe, mais nous nous contendrons d'admettre que l'explication précédente est vraie).

Cette méthode de correction itérative procède de la manière suivante :

1. On fait une estimation initiale de l'orbite grâce au système linéaire.
2. On trouve un intervalle de v_{y0} tel que les v_x au premier retour ne soit pas du même signe. Cela nous permet de mettre en place une méthode de résolution par dichotomie.
3. Et pour finir, on met en place une méthode de dichotomie dont la variable de tolérance s'appliquera à $v_x(t_{1^{er}\text{retour}})$.

Appliquée à notre cas, la méthode renvoie $v_{y0} = -21,898965265005835 \text{ m.s}^{-1}$. On peut ainsi tracer l'orbite obtenue, ce qui donne

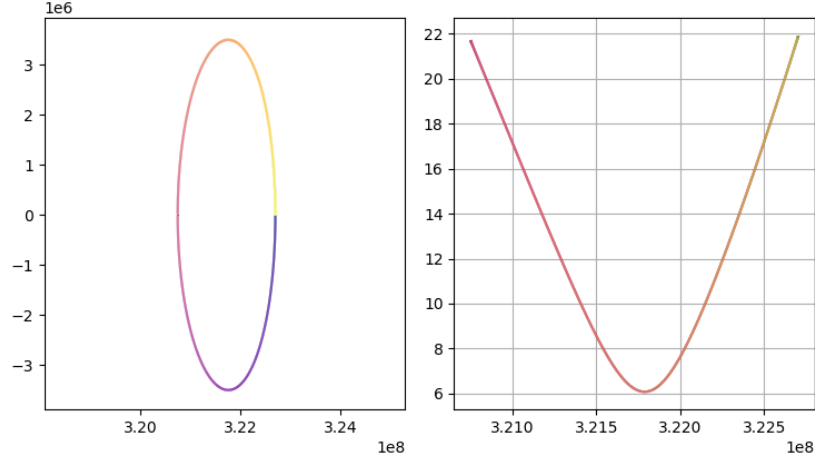


FIGURE 5 – Trajectoire tracée en python par intégration numérique des équations non-linéaires en prenant pour conditions initiales celles mentionnées précédemment.

La condition initiale n'est pas réaliste. Les directions stables de la matrice de monodromie nous assurent cependant l'existence d'une direction de stabilité marginale (liée à la deuxième multiplicité de la valeur propre 1). Ainsi, même si la condition initiale n'est pas parfaite, on trouvera une orbite assez proche de celle désirée (disons à un $O(\delta X_0)$ près, où δX_0 est la différence en valeur absolue entre la CI réelle et la CI théorique).

4 Listings Python

Dans cette section, on regroupe l'ensemble des codes pythons développés dans cette étude, ainsi que des figures tracés à partir de ces programmes, soit dans l'optique de résoudre le problème de cette étude, soit pour comparer les méthodes d'intégrations numériques. Les codes sont séparés dans multiples fichiers importés ensuite dans un seul fichier, pour simplifier l'édition, et l'organisation.

4.1 "Hub centrale" de la base de codes

Ce code, dans le fichier nommé CR3BP.py (CR3BP signifiant *Circular Restricted 3 Body Problem*), est celui permettant de rassembler tous les codes écrit et de pouvoir les utiliser ensemble. On importe ainsi la majorité des autres codes au début de ce fichier (*cf.* la suite de la section pour ces codes). Ceux qui nous seront le plus utiles ici seront les modules implémentant Runge-Kutta 4 et ceux permettant l'affichage. La majorité de ce code est constitué d'appels à des fonctions donc implémentés dans des sous-modules de la base.

```
1 import json
2 import matplotlib.pyplot as plt
3 import matplotlib
4 from pathlib import Path
5 import csv
6 import numpy as np
7
8 import src.verlet as vt
9 import src.rk as rk
10 import src.rk4 as rk4
11 import src.VFVDP as ei
12 import src.dichotomie as dich
13 import src.affichage as affichage
14 import src.lineariser as lin
15 import src.correction_orbite_lineaire as loc
16 import src.monodromie as mon
17 import src.variete as mf
18 import src.superposition_potentielle as po
19
20
21 def __init__(systeme):
22     def barycentre(m1,m2,r):
23         return (m2*r)/(m1+m2)
24     def update_system(systeme):
25         systeme["barycentre"] = barycentre(
26             systeme["corps1"]["masse"],
27             systeme["corps2"]["masse"],
28             systeme["rayon"]
29         )
30     here = Path(__file__).parent
31     with open(here / "systems.json", "r") as f:
32         data = json.load(f)[systeme]
```

```

33     update_system(data)
34     return data
35     systeme_actif = __init__("soleil_terre")
36
37
38
39     x_L1 = dich.dichotomie(0, systeme_actif["rayon"], precision=1, systeme=
        systeme_actif, pas=10)
40
41     T = 14.99625e6
42
43     x_0 = 100000000
44     # vy0 = loc.dichotomie_vy(x_0,systeme_actif,0.0000001,init_guess_coef)
45     vy0 = -128.8143495396376
46
47
48     traj_ref = rk4.simuler_trajectoire([x_0+x_L1, 0, 0, vy0], 5*T, 1000,
        systeme_actif)
49     affichage_trajectoire_toutes_variables_dans_le_temps(
50         traj_ref,
51         systeme_actif,
52         100,
53         "plasma"
54     )
55     plt.show()
56
57
58     system_period = 2*np.pi/po.omega(systeme_actif)
59     dim = [systeme_actif["rayon"], systeme_actif["rayon"], systeme_actif["
        rayon"]]/system_period, systeme_actif["rayon"]]/system_period]
60     traj_periodic = rk4.simuler_trajectoire([x_0+x_L1, 0, 0, vy0], T, 100,
        systeme_actif)
61     lambda_s = np.array([-8.34e-1,5.52e-1,-4.55e-7,2.58e-7])
62     lambda_i = np.array([-8.34e-1,-5.52e-1,4.55e-7,2.58e-7])
63     sampling = 25
64
65
66     varietes_instables_pos = mf.echantillonnage_variete_instable(
67         lambda_i,
68         dim,
69         1.8*T,
70         100,
71         systeme_actif,
72         traj_periodic,
73         sampling
74     )
75     varietes_instables_neg = mf.echantillonnage_variete_instable(

```

```

76     (-1)*lambda_i ,
77     dim,
78     1.5*T,
79     100,
80     systeme_actif ,
81     traj_periodic ,
82     sampling
83 )
84 varietes_stables_pos = mf.echantillonnage_variete_stable(
85     lambda_s ,
86     dim,
87     1.8*T,
88     100,
89     systeme_actif ,
90     traj_periodic ,
91     sampling
92 )
93 varietes_stables_neg = mf.echantillonnage_variete_stable(
94     (-1)*lambda_s ,
95     dim,
96     1.5*T,
97     100,
98     systeme_actif ,
99     traj_periodic ,
100    sampling
101 )
102
103
104 manifolds = varietes_stables_pos + varietes_stables_neg +
105             varietes_instables_pos + varietes_instables_neg
106 affichage.affichage_trajectoires_empilees(manifolds,systeme_actif)
107 plt.show()

```

4.2 Codes d'implémentations de méthodes d'intégration numérique

Dû à la sensibilité du système (système chaotique), le schéma Euler Explicite n'est pas viable. Nous optons alors pour un schéma Runge-Kutta, plus efficace, mais présentant des dérives sur la durée importante. Le schéma Runge-Kutta 4 se prouve lui d'une bien plus grande efficacité, maintenant les invariants du système à une grande précision durant les calculs des trajectoires. Ci-après sont disponibles les codes implémentant chacune de ces méthodes, ainsi que celle dite de *Verlet* modifiée : elle a été adaptée avec une prédiction intermédiaire par Runge-Kutta 4 dû à la forme de nos équations qui ne fonctionnait pas pour le schéma original. Des comparaisons entre ces schémas sont disponibles après les codes dans cette même subsection.

4.2.1 Implémentation de Euler Implicite

```
1  # Velocity-First Velocity Dependant Position Integration method (cf
    Braintruffle latest video)
2  import numpy as np
3  import time
4
5  # Numerical calculations
6  def simuler_trajectoire(conditions_initiales, temps_simulation, pas, systeme)
    :
7
8      # Tableau des instants où on souhaite résoudre le système
9      dt = pas
10     t_max = temps_simulation # Durée totale de la simulation en secondes
11     t = 0
12     i = 0
13
14     # Constantes
15     G = 6.674e-11 # Constante gravitationnelle en SI
16     masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
17     masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
18     rayon_orbital = systeme["rayon"] # Distance moyenne Terre-Lune en m
19
20     barycentre_systeme = systeme["barycentre"]
21     position_corps1 = np.array([-1*barycentre_systeme, 0.]) # Position du
        corps 1
22     rayon_corps1 = systeme["corps1"]["rayon"] # Rayon du corps 1
23     position_corps2 = np.array([rayon_orbital-barycentre_systeme, 0.]) #
        Position du corps 2
24     rayon_corps2 = systeme["corps2"]["rayon"] # Rayon du corps 2
25     omega = np.sqrt(G * (masse_corps1 + masse_corps2) / np.linalg.norm(
        position_corps1 - position_corps2)**3) # 3e loi de Kepler
26     x0,y0,vx0,vy0 = conditions_initiales
27
28     print(barycentre_systeme)
29     print(position_corps1)
```

```

30 print(position_corps2)
31
32 # Equations du système
33 def distance(x,y,pPos):
34     res = np.sqrt((x - pPos[0])**2 + (y - pPos[1])**2)
35     return res
36
37 def traction_corps_x(x,y,body_pos,masse):
38     return (G*masse*(body_pos[0] - x)) / distance(x,y,body_pos)**3
39
40 def traction_corps_y(x,y,body_pos,masse):
41     return (G*masse*(body_pos[1] - y)) / distance(x,y,body_pos)**3
42
43 def equation_x(x,y,vy):
44     coriolis = 2 * omega * vy
45     centrifuge = omega**2 * x
46     traction1 = traction_corps_x(x,y,position_corps1,masse_corps1)
47     traction2 = traction_corps_x(x,y,position_corps2,masse_corps2)
48     return traction1 + traction2 + coriolis + centrifuge
49
50 def equation_y(x,y,vx):
51     coriolis = -2 * omega * vx
52     centrifuge = omega**2 * y
53     traction1 = traction_corps_y(x,y,position_corps1,masse_corps1)
54     traction2 = traction_corps_y(x,y,position_corps2,masse_corps2)
55     return traction1 + traction2 + coriolis + centrifuge
56
57 P = [np.array([x0,y0])]
58 V = [np.array([vx0,vy0])]
59
60
61 # Calculateur de temps restant (ChatGPT)
62 pas_total = int(t_max / dt)
63 taille_bloc = 200 # mesure toutes les 200 itérations
64 debut_bloc = time.perf_counter()
65 pas_bloc_realises = 0
66 moyenne_temps_par_pas = None
67 alpha = 0.2 # lissage un peu plus fort (car mesures
68             moins fréquentes)
69
70 while t < t_max:
71     t += dt
72
73     V_new = np.array([V[i][0] + equation_x(P[i][0],P[i][1],V[i][1]) *
74                     dt, V[i][1] + equation_y(P[i][0],P[i][1],V[i][0]) * dt])

```



```

74     P_new = np.array([P[i][0] + V_new[0] * dt, P[i][1] + V_new[1] * dt
75                        ])
76
77     P.append(P_new)
78     V.append(V_new)
79
80     # Calculateur de temps restant (ChatGPT)
81     pas_bloc_realises += 1
82     if pas_bloc_realises == taille_bloc:
83         now = time.perf_counter()
84         block_wall = now - debut_bloc
85         temps_par_pas = block_wall / taille_bloc
86         if moyenne_temps_par_pas is None:
87             moyenne_temps_par_pas = temps_par_pas
88         else:
89             moyenne_temps_par_pas = (1 - alpha) *
90                 moyenne_temps_par_pas + alpha * temps_par_pas
91         debut_bloc = now
92         pas_bloc_realises = 0
93         pas_realises = int(t / dt)
94         pas_restants = max(pas_total - pas_realises, 0)
95         temps_calc_restant = pas_restants * moyenne_temps_par_pas #
96             secondes
97         if temps_calc_restant < 60:
98             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
99                 temps_calc_restant:5.1f} s ", end="\r")
100         else:
101             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
102                 temps_calc_restant/60:5.2f} min", end="\r")
103
104     i += 1
105
106     print("Simulation complete. Number of steps:", len(P))
107
108     resultat = []
109     for i in range(len(P)):
110         resultat.append([P[i][0], P[i][1], V[i][0], V[i][1]])
111     return resultat

```

4.2.2 Implémentation de Runge-Kutta

```

1  ### Code d'intégration de trajectoire
2
3  import numpy as np
4  import time
5

```

```

6
7 # Numerical calculations
8 def simuler_trajetoire(conditions_initiales, temps_simulation, pas, systeme)
9 :
10     # Tableau des instants où on souhaite résoudre le système
11     dt = pas
12     t_max = temps_simulation # Durée totale de la simulation en secondes
13     t = 0
14     i = 0
15
16     # Constantes
17     G = 6.674e-11 # Constante gravitationnelle en SI
18     masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
19     masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
20     rayon_orbital = systeme["rayon"] # Distance moyenne Terre-Lune en m
21
22     barycentre_systeme = systeme["barycentre"]
23     position_corps1 = np.array([-1*barycentre_systeme, 0.]) # Position du
24         corps 1
25     rayon_corps1 = systeme["corps1"]["rayon"] # Rayon du corps 1
26     position_corps2 = np.array([rayon_orbital-barycentre_systeme, 0.]) #
27         Position du corps 2
28     rayon_corps2 = systeme["corps2"]["rayon"] # Rayon du corps 2
29     omega = np.sqrt(G * (masse_corps1 + masse_corps2) / np.linalg.norm(
30         position_corps1 - position_corps2)**3) # 3e loi de Kepler
31     x0,y0,vx0,vy0 = conditions_initiales
32
33     print(barycentre_systeme)
34     print(position_corps1)
35     print(position_corps2)
36
37     # Equations du système
38     def distance(x,y,pPos):
39         res = np.sqrt((x - pPos[0])**2 + (y - pPos[1])**2)
40         return res
41
42     def traction_corps_x(x,y,body_pos,masse):
43         return (G*masse*(body_pos[0] - x)) / distance(x,y,body_pos)**3
44
45     def traction_corps_y(x,y,body_pos,masse):
46         return (G*masse*(body_pos[1] - y)) / distance(x,y,body_pos)**3
47
48     def equation_x(x,y,vy):
49         coriolis = 2 * omega * vy
50         centrifuge = omega**2 * x
51         traction1 = traction_corps_x(x,y,position_corps1,masse_corps1)

```

```

49         traction2 = traction_corps_x(x,y,position_corps2,masse_corps2)
50         return traction1 + traction2 + coriolis + centrifuge
51
52     def equation_y(x,y,vx):
53         coriolis = -2 * omega * vx
54         centrifuge = omega**2 * y
55         traction1 = traction_corps_y(x,y,position_corps1,masse_corps1)
56         traction2 = traction_corps_y(x,y,position_corps2,masse_corps2)
57         return traction1 + traction2 + coriolis + centrifuge
58
59
60     traj = []
61     traj.append([x0,y0,vx0,vy0])
62
63
64     # Calculateur de temps restant (ChatGPT)
65     pas_total = int(t_max / dt)
66     taille_bloc = 200 # mesure toutes les 200 itérations
67     debut_bloc = time.perf_counter()
68     pas_bloc_realises = 0
69     moyenne_temps_par_pas = None
70     alpha = 0.2 # lissage un peu plus fort (car mesures
71                 moins fréquentes)
72
73     while t < t_max:
74         t += dt
75         # print("-----")
76
77         # Initial Guess
78         kx1 = traj[i][2] * dt
79         ky1 = traj[i][3] * dt
80         kvx1 = equation_x(traj[i][0],traj[i][1],traj[i][3]) * dt
81         kvy1 = equation_y(traj[i][0],traj[i][1],traj[i][2]) * dt
82
83         # Midpoints
84         x_mid = traj[i][0] + kx1/2.
85         y_mid = traj[i][1] + ky1/2.
86         vx_mid = traj[i][2] + kvx1/2.
87         vy_mid = traj[i][3] + kvy1/2.
88
89         # Better Guess
90         kx2 = vx_mid * dt
91         ky2 = vy_mid * dt
92         kvx2 = equation_x(x_mid,y_mid,vy_mid) * dt
93         kvy2 = equation_y(x_mid,y_mid,vx_mid) * dt
94

```

```

95     # Trajectoire
96     X = traj[i][0] + kx2
97     Y = traj[i][1] + ky2
98     VX = traj[i][2] + kvx2
99     VY = traj[i][3] + kvy2
100
101     traj.append([X, Y, VX, VY])
102
103
104     # Calculateur de temps restant (ChatGPT)
105     pas_bloc_realises += 1
106     if pas_bloc_realises == taille_bloc:
107         now = time.perf_counter()
108         block_wall = now - debut_bloc
109         temps_par_pas = block_wall / taille_bloc
110         if moyenne_temps_par_pas is None:
111             moyenne_temps_par_pas = temps_par_pas
112         else:
113             moyenne_temps_par_pas = (1 - alpha) *
114                 moyenne_temps_par_pas + alpha * temps_par_pas
115         debut_bloc = now
116         pas_bloc_realises = 0
117         pas_realises = int(t / dt)
118         pas_restants = max(pas_total - pas_realises, 0)
119         temps_calc_restant = pas_restants * moyenne_temps_par_pas #
120             secondes
121         if temps_calc_restant < 60:
122             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
123                 temps_calc_restant:5.1f} s ", end="\r")
124         else:
125             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
126                 temps_calc_restant/60:5.2f} min", end="\r")
127
128     i += 1
129
130     print("Simulation complete. Number of steps:", len(traj))
131     return traj

```

4.2.3 Implémentation de Runge-Kutta 4

```

1  ### Code d'intégration de trajectoire
2
3  import numpy as np
4  import time
5
6
7  # Numerical calculations

```

```

8 def simuler_trajectoire(conditions_initiales, temps_simulation, pas, systeme,
9     inverse_temps=False):
10
11     if inverse_temps:
12         pas = -pas
13         temps_simulation = -temps_simulation
14
15     # Tableau des instants où on souhaite résoudre le système
16     dt = pas
17     t_max = temps_simulation # Durée totale de la simulation en secondes
18     t = 0
19     i = 0
20
21     # Constantes
22     G = 6.674e-11 # Constante gravitationnelle en SI
23     masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
24     masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
25     rayon_orbital = systeme["rayon"] # Distance moyenne Terre-Lune en m
26
27     barycentre_systeme = systeme["barycentre"]
28     position_corps1 = np.array([-1*barycentre_systeme, 0.]) # Position du
29         corps 1
30     rayon_corps1 = systeme["corps1"]["rayon"] # Rayon du corps 1
31     position_corps2 = np.array([rayon_orbital-barycentre_systeme, 0.]) #
32         Position du corps 2
33     rayon_corps2 = systeme["corps2"]["rayon"] # Rayon du corps 2
34     omega = np.sqrt(G * (masse_corps1 + masse_corps2) / np.linalg.norm(
35         position_corps1 - position_corps2)**3) # 3e loi de Kepler
36     x0,y0,vx0,vy0 = conditions_initiales
37
38     print(barycentre_systeme)
39     print(position_corps1)
40     print(position_corps2)
41
42     # Equations du système
43     def distance(x,y,pPos):
44         res = np.sqrt((x - pPos[0])**2 + (y - pPos[1])**2)
45         return res
46
47     def traction_corps_x(x,y,body_pos,masse):
48         return (G*masse*(body_pos[0] - x)) / distance(x,y,body_pos)**3
49
50     def traction_corps_y(x,y,body_pos,masse):
51         return (G*masse*(body_pos[1] - y)) / distance(x,y,body_pos)**3
52
53     def equation_x(x,y,vy):
54         coriolis = 2 * omega * vy

```

```

51     centrifuge = omega**2 * x
52     traction1 = traction_corps_x(x,y,position_corps1,masse_corps1)
53     traction2 = traction_corps_x(x,y,position_corps2,masse_corps2)
54     return traction1 + traction2 + coriolis + centrifuge
55
56 def equation_y(x,y,vx):
57     coriolis = -2 * omega * vx
58     centrifuge = omega**2 * y
59     traction1 = traction_corps_y(x,y,position_corps1,masse_corps1)
60     traction2 = traction_corps_y(x,y,position_corps2,masse_corps2)
61     return traction1 + traction2 + coriolis + centrifuge
62
63
64 P = [np.array([x0,y0])]
65 V = [np.array([vx0,vy0])]
66
67 def F(P,V):
68     return np.array([equation_x(P[0],P[1],V[1]), equation_y(P[0],P[1],
69                                     V[0])])
70
71 # Calculateur de temps restant (ChatGPT)
72 pas_total = int(t_max / dt)
73 taille_bloc = 200 # mesure toutes les 200 itérations
74 debut_bloc = time.perf_counter()
75 pas_bloc_realises = 0
76 moyenne_temps_par_pas = None
77 alpha = 0.2 # lissage un peu plus fort (car mesures
78             moins fréquentes)
79
80 while abs(t) < abs(t_max):
81     t += dt
82     # print("-----")
83
84     k1 = F(P[i], V[i])
85     k2 = F(
86         P[i] + V[i] * dt/2,
87         V[i] + k1 * dt/2
88     )
89     k3 = F(
90         P[i] + (V[i]*dt/2) + (k1*(dt/2)**2),
91         V[i] + (k2*dt/2)
92     )
93     k4 = F(
94         P[i] + (dt*V[i]) + (k2*dt**2/2),
95         V[i] + k3*dt
96     )

```

```

96     P.append(P[i] + dt*V[i] + dt**2/6*(k1 + k2 + k3))
97     V.append(V[i] + dt/6*(k1 + 2*k2 + 2*k3 + k4))
98
99
100
101     # Calculateur de temps restant (ChatGPT)
102     pas_bloc_realises += 1
103     if pas_bloc_realises == taille_bloc:
104         now = time.perf_counter()
105         block_wall = now - debut_bloc
106         temps_par_pas = block_wall / taille_bloc
107         if moyenne_temps_par_pas is None:
108             moyenne_temps_par_pas = temps_par_pas
109         else:
110             moyenne_temps_par_pas = (1 - alpha) *
111                 moyenne_temps_par_pas + alpha * temps_par_pas
112         debut_bloc = now
113         pas_bloc_realises = 0
114         pas_realises = int(t / dt)
115         pas_restants = max(pas_total - pas_realises, 0)
116         temps_calc_restant = pas_restants * moyenne_temps_par_pas #
117             secondes
118         if temps_calc_restant < 60:
119             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
120                 temps_calc_restant:5.1f} s ", end="\r")
121         else:
122             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
123                 temps_calc_restant/60:5.2f} min", end="\r")
124
125     i += 1
126
127     print("Simulation complete. Number of steps:", len(P))
128
129     resultat = []
130     for i in range(len(P)):
131         resultat.append(np.array([P[i][0], P[i][1], V[i][0], V[i][1]]))
132     return resultat

```

4.2.4 Implémentation de Verlet

```

1 import numpy as np
2 import time
3
4 # Numerical calculations
5 def simuler_trajectoire(conditions_initiales, temps_simulation, pas, systeme)
6     :

```

```

7 # Tableau des instants où on souhaite résoudre le système
8 dt = pas
9 t_max = temps_simulation # Durée totale de la simulation en secondes
10 t = 0
11 i = 0
12
13 # Constantes
14 G = 6.674e-11 # Constante gravitationnelle en SI
15 masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
16 masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
17 rayon_orbital = systeme["rayon"] # Distance moyenne Terre-Lune en m
18
19 barycentre_systeme = systeme["barycentre"]
20 position_corps1 = np.array([-1*barycentre_systeme, 0.]) # Position du
    corps 1
21 rayon_corps1 = systeme["corps1"]["rayon"] # Rayon du corps 1
22 position_corps2 = np.array([rayon_orbital-barycentre_systeme, 0.]) #
    Position du corps 2
23 rayon_corps2 = systeme["corps2"]["rayon"] # Rayon du corps 2
24 omega = np.sqrt(G * (masse_corps1 + masse_corps2) / np.linalg.norm(
    position_corps1 - position_corps2)**3) # 3e loi de Kepler
25 x0,y0,vx0,vy0 = conditions_initiales
26
27 print(barycentre_systeme)
28 print(position_corps1)
29 print(position_corps2)
30
31 # Equations du système
32 def distance(x,y,pPos):
33     res = np.sqrt((x - pPos[0])**2 + (y - pPos[1])**2)
34     return res
35
36 def traction_corps_x(x,y,body_pos,masse):
37     return (G*masse*(body_pos[0] - x)) / distance(x,y,body_pos)**3
38
39 def traction_corps_y(x,y,body_pos,masse):
40     return (G*masse*(body_pos[1] - y)) / distance(x,y,body_pos)**3
41
42 def equation_x(x,y,vx):
43     coriolis = 2 * omega * vy
44     centrifuge = omega**2 * x
45     traction1 = traction_corps_x(x,y,position_corps1,masse_corps1)
46     traction2 = traction_corps_x(x,y,position_corps2,masse_corps2)
47     return traction1 + traction2 + coriolis + centrifuge
48
49 def equation_y(x,y,vx):
50     coriolis = -2 * omega * vx

```



```

51     centrifuge = omega**2 * y
52     traction1 = traction_corps_y(x,y,position_corps1,masse_corps1)
53     traction2 = traction_corps_y(x,y,position_corps2,masse_corps2)
54     return traction1 + traction2 + coriolis + centrifuge
55
56 def verlet_step(P,V):
57     return [equation_x(P[0],P[1],V[1]), equation_y(P[0],P[1],V[0])]
58
59
60 P = [np.array([x0,y0])]
61 V = [np.array([vx0,vy0])]
62
63
64 # Calculateur de temps restant (ChatGPT)
65 pas_total = int(t_max / dt)
66 taille_bloc = 200 # mesure toutes les 200 itérations
67 debut_bloc = time.perf_counter()
68 pas_bloc_realises = 0
69 moyenne_temps_par_pas = None
70 alpha = 0.2 # lissage un peu plus fort (car mesures
    moins fréquentes)
71
72
73 while t < t_max:
74     t += dt
75
76     # if (i % 1000) == 0:
77     #     time.sleep(0.3)
78     #     print([P[i],V[i]])
79     P.append(P[i] + V[i]*dt + 0.5*np.array(verlet_step(P[i],V[i]))*dt
        **2)
80
81     # Approximation de V[i+1] pour f(P[i+1],V[i+1])
82     kx1 = V[i][0] * dt
83     ky1 = V[i][1] * dt
84     kvx1 = equation_x(P[i][0],P[i][1],V[i][1]) * dt
85     kvy1 = equation_y(P[i][0],P[i][1],V[i][0]) * dt
86
87     # Midpoints
88     x_mid = P[i][0] + kx1/2.
89     y_mid = P[i][1] + ky1/2.
90     vx_mid = V[i][0] + kvx1/2.
91     vy_mid = V[i][1] + kvy1/2.
92
93     kvx2 = equation_x(x_mid,y_mid,vy_mid) * dt
94     kvy2 = equation_y(x_mid,y_mid,vx_mid) * dt
95

```

```

96     VX = V[i][0] + kvx2
97     VY = V[i][1] + kvy2
98
99     V.append(V[i] + 0.5*dt*(np.array(verlet_step(P[i],V[i])) + np.
        array(verlet_step(P[i+1],np.array([VX,VY])))))
100
101
102     # Calculateur de temps restant (ChatGPT)
103     pas_bloc_realises += 1
104     if pas_bloc_realises == taille_bloc:
105         now = time.perf_counter()
106         block_wall = now - debut_bloc
107         temps_par_pas = block_wall / taille_bloc
108         if moyenne_temps_par_pas is None:
109             moyenne_temps_par_pas = temps_par_pas
110         else:
111             moyenne_temps_par_pas = (1 - alpha) *
                moyenne_temps_par_pas + alpha * temps_par_pas
112         debut_bloc = now
113         pas_bloc_realises = 0
114         pas_realises = int(t / dt)
115         pas_restants = max(pas_total - pas_realises, 0)
116         temps_calc_restant = pas_restants * moyenne_temps_par_pas #
            secondes
117         if temps_calc_restant < 60:
118             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
                temps_calc_restant:5.1f} s ", end="\r")
119         else:
120             print(f"{100*pas_realises/pas_total:6.2f}% | restant ≈ {
                temps_calc_restant/60:5.2f} min", end="\r")
121
122     i += 1
123
124     print("Simulation complete. Number of steps:", len(P))
125     resultat = []
126     for i in range(len(P)):
127         resultat.append([P[i][0],P[i][1],V[i][0],V[i][1]])
128     return resultat

```

4.2.5 Comparaison de performance de différents schémas

Remarque utile sur la lecture des graphes qui suivent : Pour illustrer l'évolution du temps sans avoir recourt à un axe, ou une flèche sur les courbes, nous avons opté pour un gradient de couleur : le temps évolue du foncé au clair (le plus souvent, du noir, au violet puis rouge, orange et enfin jaune).

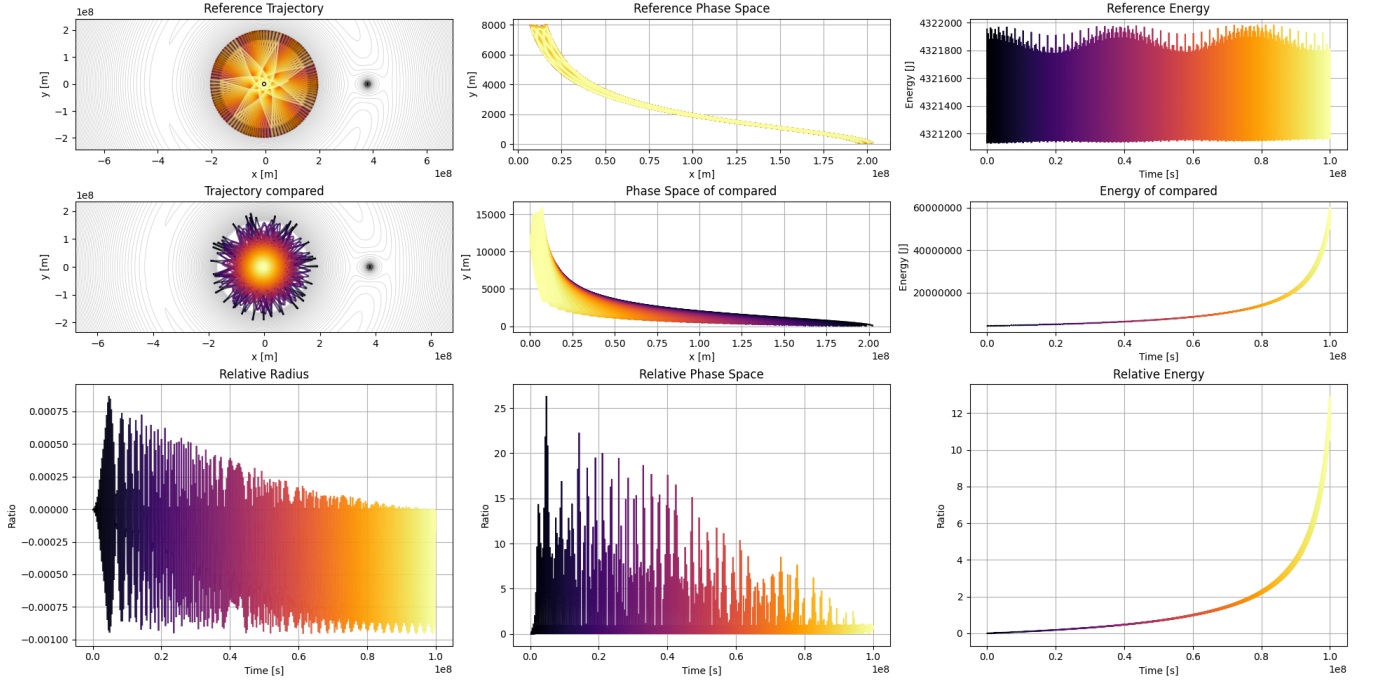


FIGURE 6 – Comparaison entre la méthode Verlet et la méthode Euler Implicite

Cette comparaison nous permet d'observer que la variation de l'énergie, qui est l'intégrale du mouvement que l'on considère, est bornée dans la méthode de Verlet. On utilise alors Verlet comme point d'étude pour comparer avec les méthodes RK et RK4. La méthode d'Euler Implicite est évidemment oubliée rien qu'à la vue de la trajectoire obtenue.

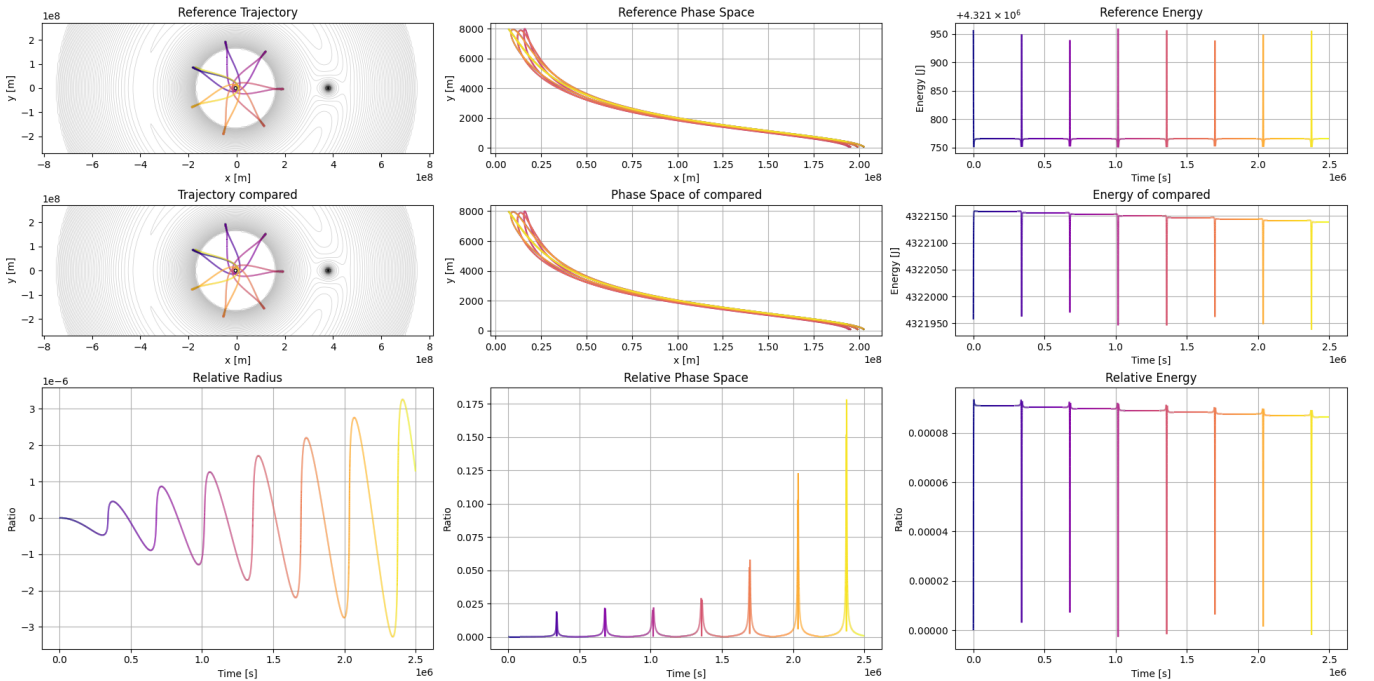


FIGURE 7 – Comparaison entre la méthode Verlet et la méthode Runge-Kutta

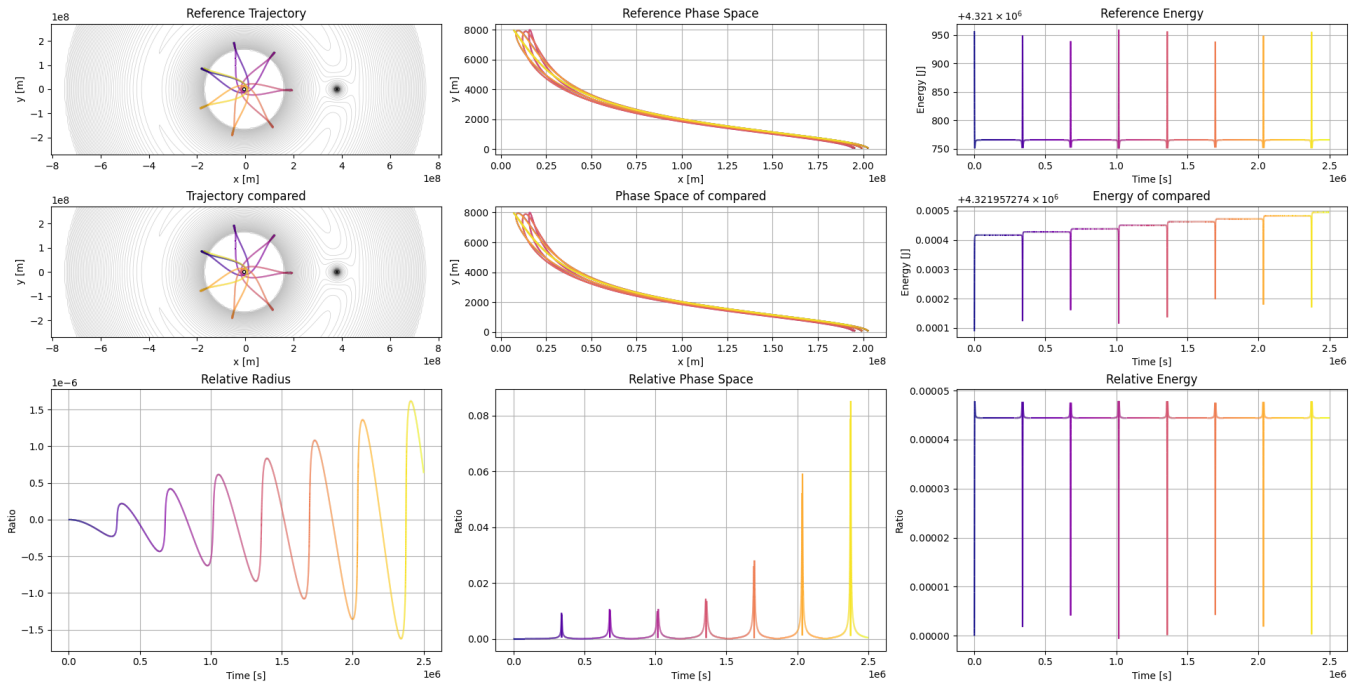


FIGURE 8 – Comparaison entre la méthode Verlet et la méthode Runge-Kutta 4

On observe alors que RK et RK4 diffèrent du résultat donné par le schéma de Verlet à un ordre près égal pour les deux (RK4 étant visiblement 2x plus précis que RK). Cependant, la méthode de Verlet prend en moyenne 150% à 200% plus de temps que RK4. Cela explique pourquoi nous faisons tous nos calculs avec RK4.

4.3 Méthodes d’affichage

Pour répondre à plusieurs besoins (affichage de la trajectoire, de l’espace de phase, d’une tranche de l’espace de phase, de l’énergie, de comparaison entre deux trajectoires ou entre deux méthodes, d’afficher plusieurs trajectoires, *etc...*), une bonne quantité de fonctions utilitaires d’affichage ont été développées, bien que leur mode de fonctionnement soit généralement le même. Ce fonctionnement en question est d’initialiser un layout qui donne la mise en page de la fenêtre d’affichage. On remplit ensuite chaque sous-fenêtre avec le graphe souhaité (en utilisant des fonctions utilitaires permettant de traiter l’information brute de la trajectoire en informations utiles).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.collections import LineCollection
4 from matplotlib.colors import Normalize
5 import src.espace_de_phase as ps
6 import src.superposition_potentielle as uo
7 import src.energie as en
8 import src.difference_relative as rd
9
10
11
12 ### FONCTION CHATGPT POUR FORCER LE DÉCALAGE
13 import matplotlib.ticker as mticker
14
15 def forcer_echelle_decalage(ax):
16     fmt = mticker.ScalarFormatter(useMathText=True)
17
18     fmt.set_useOffset(True)          # force le "+4.322e6"
19     fmt.set_scientific(False)        # interdit le "*10^6"
20
21     ax.yaxis.set_major_formatter(fmt)
22 ### FIN DU CODE CHATGPT
23
24
25 def sous_graphique_autonome(axes, etiquettes_graph):
26     for nom, ax in axes.items():
27         ax.set_title(etiquettes_graph[nom]["nom"])
28         ax.set_xlabel(etiquettes_graph[nom]["xlabel"])
29         ax.set_ylabel(etiquettes_graph[nom]["ylabel"])
30
31
32 def tracer_sur_ax(ax, graph, color_palette, display_precision=10, isTraj=False):
33     traj_arr = np.array(graph)
34     x = traj_arr[:, display_precision, 0]
35     y = traj_arr[:, display_precision, 1]
36
37     points = np.column_stack([x, y]).reshape(-1, 1, 2)
```

```

38     segments = np.concatenate([points[:-1], points[1:]], axis=1)
39
40     t_idx = np.arange(len(segments))
41     norm = Normalize(t_idx.min(), t_idx.max())
42
43     lc = LineCollection(segments, cmap=color_palette, norm=norm)
44     lc.set_array(t_idx)
45     lc.set_linewidth(1.5)
46
47     if isTraj:
48         ax.set_aspect("equal", adjustable="datalim")
49
50     ax.add_collection(lc)
51     ax.autoscale()
52     ax.autoscale_view()
53
54
55
56 def tracer_sur_ax_corps(ax, systeme_actif):
57     ax.add_patch(
58         plt.Circle(
59             [-systeme_actif["barycentre"], 0],
60             systeme_actif["corps1"]["rayon"],
61             color="black", fill=False)
62     )
63     ax.add_patch(
64         plt.Circle(
65             [systeme_actif["rayon"]-systeme_actif["barycentre"], 0],
66             systeme_actif["corps2"]["rayon"],
67             color="black", fill=False)
68     )
69
70
71 def tracer_potentiel(ax, systeme_actif, pmin=20, steps=250, levels=25,
72                     colors="k", linewidths=0.5, alpha=0.7):
73     # freeze current limits
74     xlim = ax.get_xlim()
75     ylim = ax.get_ylim()
76
77     X, Y, Z = uo.equipotentielles(systeme_actif, steps, pmin=pmin, pmax
78                                   =100)
79     ax.contour(
80         X, Y, Z,
81         levels=levels,
82         colors=colors,
83         linewidths=linewidths,
84         alpha=alpha,

```

```

84         linestyle="solid"
85     )
86
87     # restore + lock
88     ax.set_xlim(xlim)
89     ax.set_ylim(ylim)
90     ax.set_autoscale_on(False)
91
92
93
94 def affichage_trajectoire_simple(traj, systeme_actif, pas_temps=10,
95     color_palette="plasma"):
96     disposition = [
97         ["traj", "traj", "phase", "phase"],
98         ["traj", "traj", "phase", "phase"],
99         ["traj", "traj", "phase", "phase"],
100        ["energie", "energie", "energie", "energie"]
101    ]
102    fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),
103        constrained_layout=True)
104
105    tracer_sur_ax(axes["traj"], traj, color_palette, isTraj=True)
106    tracer_sur_ax_corps(axes["traj"], systeme_actif)
107    tracer_potentiel(axes["traj"], systeme_actif, pmin=30, levels=50, alpha
108        =0.2)
109
110    axes["phase"].grid(True)
111    tracer_sur_ax(axes["phase"], trajectoire_vitesse(traj), color_palette,
112        isTraj=True)
113
114    axes["energie"].grid(True)
115    tracer_sur_ax(axes["energie"], en.energie(traj, systeme_actif, pas_temps)
116        , color_palette)
117
118    labels = {
119        "traj": {
120            "nom": "Trajectoire",
121            "xlabel": "x [m]",
122            "ylabel": "y [m]",
123        },
124        "phase": {
125            "nom": "Espace des vitesses",
126            "xlabel": "v_x [m/s]",
127            "ylabel": "v_y [m/s]",
128        },
129        "energie": {
130            "nom": "Énergie",

```

```

126         "xlabel": "Temps [s]",
127         "ylabel": "Énergie [J]",
128     }
129 }
130 sous_graphique_autonome(axes, labels)
131
132
133
134 def comparaison_trois_trajectoires(traj1, traj2, traj3, systeme_actif,
135     pas_temps=10, color_palette="plasma"):
136     disposition = [
137         ["traj1", "phase1", "energie1", "energie1"],
138         ["traj2", "phase2", "energie2", "energie2"],
139         ["traj3", "phase3", "energie3", "energie3"]
140     ]
141     fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),
142         constrained_layout=True)
143
144     tracer_sur_ax(axes["traj1"], traj1, color_palette, isTraj=True)
145     tracer_sur_ax_corps(axes["traj1"], systeme_actif)
146     tracer_potentiel(axes["traj1"], systeme_actif, pmin=30, levels=50, alpha
147         =0.2)
148
149     tracer_sur_ax(axes["traj2"], traj2, color_palette, isTraj=True)
150     tracer_sur_ax_corps(axes["traj2"], systeme_actif)
151     tracer_potentiel(axes["traj2"], systeme_actif, pmin=30, levels=50, alpha
152         =0.2)
153
154     tracer_sur_ax(axes["traj3"], traj3, color_palette, isTraj=True)
155     tracer_sur_ax_corps(axes["traj3"], systeme_actif)
156     tracer_potentiel(axes["traj3"], systeme_actif, pmin=30, levels=50, alpha
157         =0.2)
158
159     axes["phase1"].grid(True)
160     tracer_sur_ax(axes["phase1"], ps.diag_espace_de_phase(traj1),
161         color_palette)
162
163     axes["phase2"].grid(True)
164     tracer_sur_ax(axes["phase2"], ps.diag_espace_de_phase(traj2),
165         color_palette)
166
167     axes["phase3"].grid(True)
168     tracer_sur_ax(axes["phase3"], ps.diag_espace_de_phase(traj3),
169         color_palette)
170
171     axes["energie1"].grid(True)
172     tracer_sur_ax(axes["energie1"], en.energie(traj1, systeme_actif,

```



```

    pas_temps), color_palette)
165 # tracer_sur_ax(axes["energie1"], en.rayon_dans_le_temps(traj1,
    pas_temps), "viridis")
166
167 axes["energie2"].grid(True)
168 tracer_sur_ax(axes["energie2"], en.energie(traj2, systeme_actif,
    pas_temps), color_palette)
169 # tracer_sur_ax(axes["energie2"], en.rayon_dans_le_temps(traj2,
    pas_temps), "viridis")
170
171 axes["energie3"].grid(True)
172 tracer_sur_ax(axes["energie3"], en.energie(traj3, systeme_actif,
    pas_temps), color_palette)
173 # tracer_sur_ax(axes["energie3"], en.rayon_dans_le_temps(traj3,
    pas_temps), "viridis")
174
175 for k in ["energie1", "energie2", "energie3"]:
176     forcer_echelle_decalage(axes[k])
177
178
179 def comparaison_traj_ref(traj_ref, traj_comp, systeme_actif, pas_temps=10,
    color_palette="plasma"):
180     disposition = [
181         ["traj_ref", "phase_ref", "energie_ref"],
182         ["traj_comp", "phase_comp", "energie_comp"],
183         ["rayon_relatif", "phase_relative", "energie_relative"],
184         ["rayon_relatif", "phase_relative", "energie_relative"]
185     ]
186     fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),
        constrained_layout=True)
187
188     rayon_ref = en.rayon_dans_le_temps(traj_ref, pas_temps)
189     rayon_comp = en.rayon_dans_le_temps(traj_comp, pas_temps)
190
191     phase_ref = ps.diag_espace_de_phase(traj_ref)
192     phase_comp = ps.diag_espace_de_phase(traj_comp)
193
194     energie_ref = en.energie(traj_ref, systeme_actif, pas_temps)
195     energie_comp = en.energie(traj_comp, systeme_actif, pas_temps)
196
197     tracer_sur_ax(axes["traj_ref"], traj_ref, color_palette, isTraj=True)
198     tracer_sur_ax_corps(axes["traj_ref"], systeme_actif)
199     tracer_potentiel(axes["traj_ref"], systeme_actif, pmin=30, levels=50,
        alpha=0.2)
200
201     tracer_sur_ax(axes["traj_comp"], traj_comp, color_palette, isTraj=True)
202     tracer_sur_ax_corps(axes["traj_comp"], systeme_actif)

```

```

203 tracer_potentiel(axes["traj_comp"],systeme_actif,pmin=30,levels=50,
    alpha=0.2)
204
205 axes["phase_ref"].grid(True)
206 tracer_sur_ax(axes["phase_ref"],phase_ref,color_palette)
207
208 axes["phase_comp"].grid(True)
209 tracer_sur_ax(axes["phase_comp"],phase_comp,color_palette)
210
211 axes["energie_ref"].grid(True)
212 tracer_sur_ax(axes["energie_ref"],energie_ref,color_palette)
213 # tracer_sur_ax(axes["energie_ref"],en.rayon_dans_le_temps(traj_ref,
    pas_temps),"viridis")
214
215 axes["energie_comp"].grid(True)
216 tracer_sur_ax(axes["energie_comp"],energie_comp,color_palette)
217 # tracer_sur_ax(axes["energie_comp"],en.rayon_dans_le_temps(traj_comp,
    pas_temps),"viridis")
218
219 axes["rayon_relatif"].grid(True)
220 tracer_sur_ax(axes["rayon_relatif"],rd.difference_relative_1D(
    rayon_ref, rayon_comp, pas_temps),color_palette)
221
222 axes["phase_relative"].grid(True)
223 tracer_sur_ax(axes["phase_relative"],rd.difference_relative_2D(
    phase_ref, phase_comp, pas_temps),color_palette)
224
225 axes["energie_relative"].grid(True)
226 tracer_sur_ax(axes["energie_relative"],rd.difference_relative_1D(
    energie_ref, energie_comp, pas_temps),color_palette)
227
228 labels = {
229     "traj_ref": {
230         "nom": "Trajectoire 1",
231         "xlabel": "x [m]",
232         "ylabel": "y [m]",
233     },
234     "phase_ref": {
235         "nom": "Espace de phase de la trajectoire 1",
236         "xlabel": "v_x [m/s]",
237         "ylabel": "v_y [m/s]",
238     },
239     "energie_ref": {
240         "nom": "Énergie de la trajectoire 1",
241         "xlabel": "Temps [s]",
242         "ylabel": "Énergie [J]",
243     },

```

```

244     "traj_comp": {
245         "nom": "Trajectoire 2",
246         "xlabel": "x [m]",
247         "ylabel": "y [m]",
248     },
249     "phase_comp": {
250         "nom": "Espace de phase de la trajectoire 2",
251         "xlabel": "v_x [m/s]",
252         "ylabel": "v_y [m/s]",
253     },
254     "energie_comp": {
255         "nom": "Énergie de la trajectoire 2",
256         "xlabel": "Temps [s]",
257         "ylabel": "Énergie [J]",
258     },
259     "rayon_relatif": {
260         "nom": "Différence relative de rayon",
261         "xlabel": "Temps [s]",
262         "ylabel": "Rapport relatif",
263     },
264     "phase_relative": {
265         "nom": "Différence relative de l'état dans l'espace de phase
266             en norme.",
267         "xlabel": "Temps [s]",
268         "ylabel": "Rapport relatif",
269     },
270     "energie_relative": {
271         "nom": "Différence relative de l'énergie",
272         "xlabel": "Temps [s]",
273         "ylabel": "Rapport relatif",
274     },
275 }
276
277 sous_graphique_autonome(axes, labels)
278
279 for k in ["energie_ref", "energie_comp", "energie_relative"]:
280     forcer_echelle_decalage(axes[k])
281
282 def espaces_de_phase(traj, systeme_actif, color_palette):
283     disposition = [
284         ["traj", "phase1", "phase2"],
285         ["traj", "phase1", "phase2"],
286         ["phase", "phase1", "phase2"],
287         ["phase", "phase1", "phase2"]
288     ]
289     fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),

```

```

        constrained_layout=True)
290 tracer_sur_ax(axes["traj"],traj,color_palette,isTraj=True)
291 tracer_sur_ax_corps(axes["traj"],systeme_actif)
292 tracer_potentiel(axes["traj"],systeme_actif,pmin=30,levels=50,alpha
    =0.2)
293
294 axes["phase"].grid(True)
295 tracer_sur_ax(axes["phase"],ps.diag_espace_de_phase(traj),
    color_palette)
296
297 axes["phase1"].grid(True)
298 tracer_sur_ax(axes["phase1"],ps.coupe_x_espace_de_phase(traj),
    color_palette)
299
300 axes["phase2"].grid(True)
301 tracer_sur_ax(axes["phase2"],ps.coupe_y_espace_de_phase(traj),
    color_palette)
302
303
304
305 def affichage_trajectoire_origine_relative(traj,systeme_actif,pas_temps
=10,color_palette="plasma"):
306     disposition = [
307         ["traj","traj","rayon_relatif","rayon_relatif"],
308         ["traj","traj","rayon_relatif","rayon_relatif"],
309         ["energie", "energie", "energie", "energie"]
310     ]
311     fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),
        constrained_layout=True)
312     tracer_sur_ax(axes["traj"],traj,color_palette,isTraj=True)
313     tracer_sur_ax_corps(axes["traj"],systeme_actif)
314     tracer_potentiel(axes["traj"],systeme_actif,pmin=98,levels=50,steps
        =4000,alpha=0.2)
315
316     # axes["phase"].grid(True)
317     # tracer_sur_ax(axes["phase"],ps.diag_espace_de_phase(traj),
        color_palette)
318
319     axes["energie"].grid(True)
320     tracer_sur_ax(axes["energie"],en.energie(traj,systeme_actif,pas_temps)
        ,color_palette)
321
322     traj_origine_cte = [traj[0]+[0.1,0,0,0] for i in range(len(traj))]
323     origine_rayon = en.rayon_dans_le_temps(traj_origine_cte,pas_temps)
324     rayon_traj = en.rayon_dans_le_temps(traj,pas_temps)
325     orbite_degenerescence = rd.difference_relative_1D(rayon_traj,
        origine_rayon, pas_temps)

```

```

326 axes["rayon_relatif"].grid(True)
327 tracer_sur_ax(axes["rayon_relatif"], orbite_degenerescence,
328               color_palette)
329
330 labels = {
331     "traj": {
332         "nom": "Trajectoire",
333         "xlabel": "x [m]",
334         "ylabel": "y [m]",
335     },
336     # "phase": {
337     #     "nom": "Espace de phase en norme",
338     #     "xlabel": "rayon [m]",
339     #     "ylabel": "vitesse [m/s]",
340     # },
341     "energie": {
342         "nom": "Énergie",
343         "xlabel": "Temps [s]",
344         "ylabel": "Énergie [J]",
345     },
346     "rayon_relatif": {
347         "nom": "Distance à l'origine spatiale",
348         "xlabel": "Temps [s]",
349         "ylabel": "Déplacement relatif [/]",
350     },
351 }
352 sous_graphique_autonome(axes, labels)
353
354 for k in ["energie", "rayon_relatif"]:
355     forcer_echelle_decalage(axes[k])
356
357 return orbite_degenerescence
358
359
360
361 def trajectoire_vitesse(traj):
362     traj_np = np.array(traj)
363     return traj_np[:, 2:4]
364
365 def coordinate_projection(traj, coord, pas_temps):
366     traj_np = np.array(traj)
367     traj_return = []
368     for i in range(len(traj)):
369         traj_return.append([i*pas_temps, traj_np[i][coord]])
370     return np.array(traj_return)
371

```

```

372
373
374 def affichage_trajectoires_empilees(trajs,systeme_actif,pas_temps=10,
color_palette=[]):
375     if color_palette == []:
376         for i in range(len(trajs)//2):
377             color_palette.append("plasma")
378         for i in range(len(trajs)//2):
379             color_palette.append("viridis")
380     disposition = [
381         ["traj","phase"]
382     ]
383     fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),
constrained_layout=True)
384     for i in range(len(trajs)):
385         tracer_sur_ax(axes["traj"],trajs[i],color_palette[i],isTraj=True)
386         tracer_sur_ax(axes["phase"],trajectoire_vitesse(trajs[i]),
color_palette[i],isTraj=True)
387     tracer_sur_ax_corps(axes["traj"],systeme_actif)
388     tracer_potentiel(axes["traj"],systeme_actif,pmin=98,levels=50,steps
=4000,alpha=0.2)
389
390     labels = {
391         "traj": {
392             "nom": "Trajectoire dans le plan",
393             "xlabel": "x [m]",
394             "ylabel": "y [m]",
395         },
396         "phase": {
397             "nom": "Espace des vitesse",
398             "xlabel": "vx [m/s]",
399             "ylabel": "vy [m/s]",
400         }
401     }
402     sous_graphique_autonome(axes, labels)
403
404
405
406
407 def trajectoire_toutes_variables_dans_le_temps(traj,systeme_actif,
pas_temps=10,color_palette="plasma"):
408     disposition = [
409         ["traj","x_proj"],
410         ["traj","y_proj"],
411         ["vitesse_proj","vx_proj"],
412         ["vitesse_proj","vy_proj"]
413     ]

```

```

414 fig, axes = plt.subplot_mosaic(disposition, figsize=(12, 6),
415                               constrained_layout=True)
416 tracer_sur_ax(axes["traj"], traj, color_palette, isTraj=True)
417 tracer_sur_ax_corps(axes["traj"], systeme_actif)
418 velocity = trajectoire_vitesse(traj)
419 tracer_sur_ax(axes["vitesse_proj"], velocity, color_palette, isTraj=True)
420 tracer_potentiel(axes["traj"], systeme_actif, pmin=30, levels=50, alpha
    =0.2)
421
422 tracer_sur_ax(axes["x_proj"], coordinate_projection(traj, 0, pas_temps),
    color_palette)
423 tracer_sur_ax(axes["y_proj"], coordinate_projection(traj, 1, pas_temps),
    color_palette)
424 tracer_sur_ax(axes["vx_proj"], coordinate_projection(traj, 2, pas_temps),
    color_palette)
425 tracer_sur_ax(axes["vy_proj"], coordinate_projection(traj, 3, pas_temps),
    color_palette)
426
427
428 labels = {
429     "traj": {
430         "nom": "Trajectoire dans le plan",
431         "xlabel": "x [m]",
432         "ylabel": "y [m]",
433     },
434     "vitesse_proj": {
435         "nom": "Espace des vitesse",
436         "xlabel": "vx [m/s]",
437         "ylabel": "vy [m/s]",
438     },
439     "x_proj": {
440         "nom": "Projection de la trajectoire sur x",
441         "xlabel": "Temps [s]",
442         "ylabel": "x [m]",
443     },
444     "y_proj": {
445         "nom": "Projection de la trajectoire sur y",
446         "xlabel": "Temps [s]",
447         "ylabel": "y [m]",
448     },
449     "vx_proj": {
450         "nom": "Projection de la trajectoire sur vx",
451         "xlabel": "Temps [s]",
452         "ylabel": "vx [m/s]",
453     },
454     "vy_proj": {

```

```
455         "nom": "Projection de la trajectoire sur vy",
456         "xlabel": "Temps [s]",
457         "ylabel": "vy [m/s]",
458     }
459 }
460 sous_graphique_autonome(axes, labels)
```


4.4 Affichage du potentiel apparent sur les fenêtres de trajectoires

```
1 import numpy as np
2
3
4 def distance(x,y,pPos):
5     res = np.sqrt((x - pPos[0])**2 + (y - pPos[1])**2)
6     return res
7
8 def omega(systeme):
9     G = 6.674e-11 # Constante gravitationnelle en SI
10    masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
11    position_corps1 = np.array([-1*systeme["barycentre"], 0.]) # Position
12    du corps 1
13    masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
14    position_corps2 = np.array([systeme["rayon"]-systeme["barycentre"],
15    0.]) # Position du corps 2
16
17    omega = np.sqrt(G * (masse_corps1 + masse_corps2) / np.linalg.norm(
18    position_corps1 - position_corps2)**3) # 3e loi de Kepler
19    return omega
20
21 def potentiel(x,y,systeme):
22     G = 6.674e-11 # Constante gravitationnelle en SI
23     masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
24     position_corps1 = np.array([-1*systeme["barycentre"], 0.]) # Position
25     du corps 1
26     masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
27     position_corps2 = np.array([systeme["rayon"]-systeme["barycentre"],
28     0.]) # Position du corps 2
29
30     rayon_orbital = systeme["rayon"] # Distance moyenne Terre-Lune en m
31     grav_potential_body1 = -1 * G*masse_corps1/(distance(x,y,
32     position_corps1))
33     grav_potential_body2 = -1 * G*masse_corps2/(distance(x,y,
34     position_corps2))
35     potentiel_centrifuge = -0.5 * omega(systeme)**2*(x**2+y**2)
36     resultat = grav_potential_body1 + grav_potential_body2 +
37     potentiel_centrifuge
38     return resultat
39
40 def potentiel_total(systeme,surface,pas):
41     x = np.linspace(surface[0][0], surface[0][1], pas)
42     y = np.linspace(surface[1][0], surface[1][1], pas)
43     X,Y = np.meshgrid(x,y)
44     Z = potentiel(X,Y,systeme)
45     return (X,Y,Z)
```

```

39
40 def equipotentielles(systeme,pas,pmin,pmax):
41     surface = [
42         [-2*systeme["rayon"], 2*systeme["rayon"]],
43         [-2*systeme["rayon"], 2*systeme["rayon"]]
44     ]
45     X,Y,Z = potentiel_total(systeme,surface,pas)
46     Z = np.nan_to_num(Z, nan=np.nan, posinf=np.nan, neginf=np.nan)
47     vmin, vmax = np.nanpercentile(Z, [pmin, pmax])
48     Z_plot = np.clip(Z, vmin, vmax)
49     return X,Y,Z_plot

```

4.5 Visualisations du potentiel apparent

Ce code possède des résultats plus ou moins fructueux. Les fonctions ayant pour but un affichage 3D n'ont pas abouti (résultat visuellement peu clair). On se contentera alors de la représentation au moyen des équipotentielles en 2D.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import vector
4 from matplotlib.colors import PowerNorm, LightSource
5
6 # Constantes factices
7 G = 1 # Constante gravitationnelle (unités arbitraires)
8 masse_terre = 1 # Masse de la Terre
9 masse_soleil = 10 # Masse du Soleil
10 d = 1.0 # Distance Soleil-Terre (unités arbitraires)
11 W = np.sqrt(G * (masse_soleil + masse_terre) / d**3) # Vitesse de
    rotation
12
13 # Grille autour du système Soleil-Terre
14 x = np.linspace(-1.5, 2.5, 200)
15 y = np.linspace(-1.5, 1.5, 200)
16 X, Y = np.meshgrid(x, y)
17
18 sun_dist = d*masse_terre/(masse_soleil+masse_terre)
19
20 earth_pos = vector.obj(x=d-sun_dist, y=0.0)
21 sun_pos = vector.obj(x=-sun_dist, y=0.0)
22
23 # Champ de potentiel
24 potentiel = np.zeros_like(X)
25
26 potentiel_gravitationnel = np.zeros_like(X)
27 potentiel_centrifuge = np.zeros_like(X)
28
29 for i in range(X.shape[0]):
30     for j in range(X.shape[1]):
31         r_vec = vector.obj(x=X[i, j], y=Y[i, j])
32         r_earth = (r_vec - earth_pos).rho + 1e-6
33         r_sun = (r_vec - sun_pos).rho + 1e-6
34         potentiel_gravitationnel[i, j] = -G * (masse_soleil / r_sun +
            masse_terre / r_earth)
35         potentiel_centrifuge[i, j] = 0.5 * W**2 * r_vec.rho**2
36
37 potentiel = potentiel_gravitationnel - potentiel_centrifuge
38
39
40
```

```

41 # norm = TwoSlopeNorm(vmin=potentiel.min(), vcenter=0, vmax=potentiel.max
    ())
42
43 vmin = np.percentile(potentiel, 1)
44 vmax = np.percentile(potentiel, 99)
45
46 # Traçage
47 fig, ax = plt.subplots(figsize=(10, 8))
48 cont = ax.contour(X, Y, potentiel, levels=10000, cmap='plasma', vmin=vmin,
    vmax=vmax, norm=PowerNorm(gamma=0.8))
49 plt.colorbar(cont, label='Energie Potentielle Apparente (grandeurs
    arbitraires)')
50
51 # Marquer le Soleil et la Terre
52 ax.plot(-sun_dist, 0, 'yo', label='Soleil')
53 ax.plot(d-sun_dist, 0, 'bo', label='Terre')
54
55 ax.legend()
56 ax.set_title("Visualisation du potentiel apparent")
57 ax.set_xlabel("x (non-dimensionnalisé)")
58 ax.set_ylabel("y (non-dimensionnalisé)")
59 ax.set_aspect('equal')
60 plt.grid(True)
61 plt.tight_layout()
62 plt.show()
63
64
65
66 ### ESSAIS 3D ###
67
68 def distance(x,y,pPos):
69     res = np.sqrt((x - pPos[0])**2 + (y - pPos[1])**2)
70     return res
71
72 def potentiel(x,y,systeme):
73     G = 6.674e-11 # Constante gravitationnelle en SI
74     masse_corps1 = systeme["corps1"]["masse"] # Masse de la Terre en kg
75     position_corps1 = np.array([-1*systeme["barycentre"], 0.]) # Position
        du corps 1
76     masse_corps2 = systeme["corps2"]["masse"] # Masse de la Lune en kg
77     position_corps2 = np.array([systeme["rayon"]-systeme["barycentre"],
        0.]) # Position du corps 2
78     omega = np.sqrt(G * (masse_corps1 + masse_corps2) / np.linalg.norm(
        position_corps1 - position_corps2)**3) # 3e loi de Kepler
79     rayon_orbital = systeme["rayon"] # Distance moyenne Terre-Lune en m
80
81     grav_potential_body1 = -1 * G*masse_corps1/(distance(x,y,

```

```

    position_corps1))
82 grav_potential_body2 = -1 * G*masse_corps2/(distance(x,y,
    position_corps2))
83
84 potentiel_centrifuge = -0.5 * omega**2*(x**2+y**2)
85
86 resultat = grav_potential_body1 + grav_potential_body2 +
    potentiel_centrifuge
87
88 return resultat
89
90
91 def potentiel_total(systeme,surface,pas):
92     x = np.linspace(surface[0][0], surface[0][1], pas)
93     y = np.linspace(surface[1][0], surface[1][1], pas)
94
95     X,Y = np.meshgrid(x,y)
96
97     Z = potentiel(X,Y,systeme)
98
99     return (X,Y,Z)
100
101
102 def tracer_potentiel_3d_limite(systeme, pas, pmin=2, pmax=100):
103
104     surface = [
105         [-1.8*systeme["rayon"], 1.8*systeme["rayon"]],
106         [-1.8*systeme["rayon"], 1.8*systeme["rayon"]]
107     ]
108
109     # Calculer la grille et le potentiel
110     X, Y, Z = potentiel_total(systeme, surface, pas)
111
112     # Nettoyer les infinis le cas échéant
113     Z = np.nan_to_num(Z, nan=np.nan, posinf=np.nan, neginf=np.nan)
114
115     # Choisir une fenêtre Z "raisonnable" basée sur des percentiles
116     vmin, vmax = np.nanpercentile(Z, [pmin, pmax])
117
118     # Limiter Z pour le tracé (évite les pics énormes près des singularité
119     s)
120     Z_plot = np.clip(Z, vmin, vmax)
121
122     # Figure 3D
123     fig = plt.figure(figsize=(8, 6))
124     ax = fig.add_subplot(111, projection="3d")

```

```

125 # Surface (vous pouvez ajuster rstride/cstride pour échanger détail et
    vitesse)
126 surf = ax.plot_surface(
127     X, Y, Z_plot,
128     rstride=2,
129     cstride=2,
130     linewidth=0,
131     antialiased=True,
132 )
133
134 # Barre de couleurs
135 fig.colorbar(surf, shrink=0.6, aspect=12, label=" $\Phi(x, y)$  (tronqué)")
136
137 # Étiquettes des axes
138 ax.set_xlabel("x")
139 ax.set_ylabel("y")
140 ax.set_zlabel(" $\Phi(x, y)$ ")
141
142 # Limiter l'axe z à la même plage tronquée
143 ax.set_zlim(vmin, vmax)
144
145 plt.tight_layout()
146 plt.show()
147
148
149
150
151 def tracer_potentiel_3d_opaque(systeme, pas, pmin=20, pmax=100):
152     surface = [
153         [-1.8*systeme["rayon"], 1.8*systeme["rayon"]],
154         [-1.8*systeme["rayon"], 1.8*systeme["rayon"]]
155     ]
156
157     X, Y, Z = potentiel_total(systeme, surface, pas)
158
159     # Supprimer les valeurs inf
160     Z = np.nan_to_num(Z, nan=np.nan, posinf=np.nan, neginf=np.nan)
161
162     # Limiter le potentiel à une fenêtre raisonnable
163     vmin, vmax = np.nanpercentile(Z, [pmin, pmax])
164     Z_plot = np.clip(Z, vmin, vmax)
165
166     # Centrer la palette de couleurs sur le milieu de la région tronquée (
        plus clair visuellement)
167     mid = 0.5 * (vmin + vmax)
168     norm = Normalize(vmin=vmin, vmax=vmax, clip=True)
169

```

```

170 # --- FIGURE 3D ---
171 fig = plt.figure(figsize=(10, 7))
172 ax = fig.add_subplot(111, projection='3d')
173
174 surf = ax.plot_surface(
175     X, Y, Z_plot,
176     cmap='terrain',          # or 'inferno' for strong contrast
177     norm=norm,
178     rstride=1, cstride=1,
179     linewidth=0,
180     antialiased=False,
181     shade=True,              # <--- IMPORTANT: shading ON
182     alpha=1.0,               # <--- FULL OPACITY, no see-through
183     edgecolor='none'         # <--- No transparent grille
184 )
185
186 # Barre de couleurs
187 fig.colorbar(surf, shrink=0.55, aspect=12, label=" $\Phi$  (tronqué)")
188
189 # Étiquettes des axes
190 ax.set_xlabel("x")
191 ax.set_ylabel("y")
192 ax.set_zlabel(" $\Phi$ ")
193
194 # S'assurer que les limites z correspondent au découpage
195 ax.set_zlim(vmin, vmax)
196
197 plt.tight_layout()
198 plt.show()
199
200
201
202
203 def tracer_potentiel_3d_ombrage(systeme, pas,
204                                 pmin=2, pmax=100, base_color=(0.75, 0.80,
205                                                                 0.90)):
206
207     surface = [
208         [-1.8*systeme["rayon"], 1.8*systeme["rayon"]],
209         [-1.8*systeme["rayon"], 1.8*systeme["rayon"]]
210     ]
211
212     # Calculer la grille et le potentiel
213     X, Y, Z = potentiel_total(systeme, surface, pas)
214
215     # Nettoyer les nombres invalides
216     Z = np.nan_to_num(Z, nan=np.nan, posinf=np.nan, neginf=np.nan)

```

```

216
217 # Limiter les extrêmes de Z pour éviter que les singularités n'é
      crasent le tracé
218 vmin, vmax = np.nanpercentile(Z, [pmin, pmax])
219 Z_plot = np.clip(Z, vmin, vmax)
220
221 # Construire un tableau de couleurs RGB uniforme de forme (Ny, Nx, 3)
222 ny, nx = Z_plot.shape
223 base = np.zeros((ny, nx, 3), dtype=float)
224 base[..., 0] = base_color[0]
225 base[..., 1] = base_color[1]
226 base[..., 2] = base_color[2]
227
228 # Source de lumière
229 ls = LightSource(azdeg=120, altdeg=10)
230
231 # Appliquer un ombrage à la couleur de base uniforme en utilisant la
      carte d'altitude Z_plot
232 rgb = ls.shade_rgb(base, Z_plot) # forme (Ny, Nx, 3)
233
234 # --- Tracé ---
235 fig = plt.figure(figsize=(10, 7))
236 ax = fig.add_subplot(111, projection='3d')
237
238 surf = ax.plot_surface(
239     X, Y, Z_plot,
240     facecolors=rgb,
241     rstride=1, cstride=1,
242     linewidth=0,
243     antialiased=False
244 )
245
246 ax.set_xlabel("x")
247 ax.set_ylabel("y")
248 ax.set_zlabel("Φ")
249 ax.set_zlim(vmin, vmax)
250
251 plt.tight_layout()
252 plt.show()

```


4.6 Codes utilitaires

Dans cette sous-section, on recense différents codes utilitaires, utilisés dans le traitement des données brutes fournies par les simulations de trajectoires de `rk4.py`.

4.6.1 Calcul des écarts relatifs

```
1 import numpy as np
2
3 ### Calcul la distance entre les données relativement à la taille de la
   donnée de référence
4
5 def difference_relatif_1D(ref, donnees, pas):
6     relatif = []
7     t = 0
8     if (len(ref) != len(donnees)):
9         print("Erreur : les données et la référence n'ont pas la même
              longueur.")
10        return [[0,0]]
11    for i in range(len(donnees)):
12        if (ref[i][1] == 0):
13            print("Erreur : division par 0.")
14            relatif.append([ref[i][0],0])
15        else:
16            relatif.append([ref[i][0],(donnees[i][1]-ref[i][1])/ref[i]
                           ][1]])
17        t += pas
18    return relatif
19
20 def difference_relatif_2D(ref, donnees, pas):
21     relatif = []
22     t = 0
23     if (len(ref) != len(donnees)):
24         print("Erreur : les données et la référence n'ont pas la même
              longueur.")
25        return [[0,0]]
26    for i in range(len(donnees)):
27        x = donnees[i][0]-ref[i][0]
28        y = donnees[i][1]-ref[i][1]
29        relatif.append([t,np.sqrt(x**2+y**2)/np.sqrt(ref[i][0]**2+ref[i]
               ][1]**2)])
30        t += pas
31    return relatif
```

4.6.2 Calcul de l'énergie

```

1 import src.superposition_potentielle as uo
2
3 def energie(traj,systeme_actif,pas_temps):
4     t = 0
5     e = []
6     for i in range(len(traj)):
7         x = traj[i][0]
8         y = traj[i][1]
9         vx = traj[i][2]
10        vy = traj[i][3]
11        energie_potentielle = uo.potentiel(x,y,systeme_actif)
12        energie_cinetique = 0.5*(vx**2 + vy**2)
13        e.append([t,-2*energie_potentielle-2*energie_cinetique])
14        t += pas_temps
15    return e
16
17 def rayon_dans_le_temps(traj,pas_temps):
18     t = 0
19     r = []
20     for i in range(len(traj)):
21         x = traj[i][0]
22         y = traj[i][1]
23         r.append([t,(x**2+y**2)/1e13+4.32e6])
24         t += pas_temps
25    return r

```

4.6.3 Calcul de l'espace de phase

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.collections import LineCollection
4 from matplotlib.colors import Normalize
5
6
7 # Utilitaires d'affichage
8 def diag_espace_de_phase(traj):
9     """
10     Retourne un diagramme de l'espace de flux associé à une trajectoire
11     sous la forme de deux tableaux r et v où r est la position absolue
12     et v est la vitesse absolue
13     """
14     diag = []
15     for i in range(len(traj)):
16         x = traj[i][0]
17         y = traj[i][1]
18         vx = traj[i][2]

```

```

19         vy = traj[i][3]
20         pos_abs = np.sqrt(x**2 + y**2)
21         vit_abs = np.sqrt(vx**2 + vy**2)
22         diag.append([pos_abs, vit_abs])
23     return diag
24
25 def coupe_x_espace_de_phase(traj):
26     diag = []
27     for i in range(len(traj)):
28         x = traj[i][0]
29         vx = traj[i][2]
30         diag.append([x, vx])
31     return diag
32
33 def coupe_y_espace_de_phase(traj):
34     diag = []
35     for i in range(len(traj)):
36         x = traj[i][1]
37         vx = traj[i][3]
38         diag.append([x, vx])
39     return diag

```

4.7 Calcul d'une condition initiale pour le système non-linéaire

Cette sous-section possède le code dont le fonctionnement est illustré avant le début des listings.

```
1  ### Faire le code pour la meta-dichotomie pour minimiser la vitesse en x
   lors du retour en y=0.
2  # Idée : Utiliser la position relative de la branche du retour dans l'
   espace de phase
3  # (témoigne du signe de la vitesse en x pour une raison ou une autre...
   weird)
4
5
6  import numpy as np
7  import src.rk4 as rk4
8  import matplotlib.pyplot as plt
9  import src.dichotomie as dich
10 import src.affichage as affichage
11
12
13 T = 15e6
14
15
16 def estimation_orbite_lineaire_0(amplitude, estimation_init_coef):
17     return np.array([amplitude, 0, 0, estimation_init_coef*amplitude])
18
19
20 def plus_proche_dans_index(liste, reference, index_considered):
21     if len(liste) == 0:
22         return -1
23     meilleur_i = 0
24     meilleur_diff = abs(liste[0][index_considered] - reference)
25     for i in range(len(liste)):
26         diff = abs(liste[i][index_considered] - reference)
27         if diff < meilleur_diff:
28             meilleur_i = i
29             meilleur_diff = diff
30     return meilleur_i
31
32
33 # Hypothèse : il n'y a qu'une seule branche qui repasse par y = 0 après T
   /4 et l'impulsion initiale est en v_y < 0
34 def etat_retour(traj, systeme_actif):
35     for i in range(2, len(traj)):
36         if traj[i-1][1] < 0 and traj[i][1] > 0:
37             alpha = -traj[i-1][1] / (traj[i][1] - traj[i-1][1])
38             return traj[i-1] + alpha * (traj[i] - traj[i-1])
39     affichage.affichage_trajetoire_simple(traj, systeme_actif)
40     plt.show()
```

```

41     return None
42
43
44 def trouver_intervalle(systeme, gamma_0):
45     trouve = False
46     i = 1
47     while not trouve:
48         print("Recherche n°", i)
49         perturbation = np.array([0, 0, 0, 0.01*i*gamma_0[3]])
50         traj_a = rk4.simuler_trajectoire(gamma_0+perturbation, 2*T/3, 100,
51             systeme)
52         traj_b = rk4.simuler_trajectoire(gamma_0-perturbation, 2*T/3, 100,
53             systeme)
54         affichage.comparaison_traj_ref(traj_a, traj_b, systeme)
55         plt.show()
56         x_point_a = etat_retour(traj_a, systeme)[2]
57         x_point_b = etat_retour(traj_b, systeme)[2]
58         print(x_point_a, x_point_b)
59         if x_point_a * x_point_b < 0:
60             trouve = True
61         else:
62             i += 1
63     print()
64     print("-----")
65     print("Bracket trouvé : ", gamma_0[3] - 0.01*i*gamma_0[3], gamma_0[3]
66         + 0.01*i*gamma_0[3])
67     print("-----")
68     print()
69     return [gamma_0[3] - 0.01*i*gamma_0[3], gamma_0[3] + 0.01*i*gamma_0
70         [3]]
71
72 def dichotomie_vy(amplitude, systeme, precision, estimation_init_coef):
73     x_L1 = dich.dichotomie(0, systeme["rayon"], precision=1, systeme=
74         systeme, pas=10)
75     gamma_0 = estimation_orbite_lineaire_0(amplitude, estimation_init_coef)
76         + np.array([x_L1, 0, 0, 0])
77     bracket = trouver_intervalle(systeme, gamma_0)
78     c = (bracket[0]+bracket[1])/2
79     gamma_a = gamma_0.copy()
80     gamma_b = gamma_0.copy()
81     gamma_c = gamma_0.copy()
82     gamma_a[3] = bracket[0]
83     gamma_b[3] = bracket[1]
84     gamma_c[3] = c
85     traj_a = rk4.simuler_trajectoire(gamma_a, 2*T/3, 100, systeme)
86     traj_b = rk4.simuler_trajectoire(gamma_b, 2*T/3, 100, systeme)
87     traj_c = rk4.simuler_trajectoire(gamma_c, 2*T/3, 100, systeme)

```

```

82     x_point_a = etat_retour(traj_a, systeme)[2]
83     print("A renvoyé état a")
84     x_point_b = etat_retour(traj_b, systeme)[2]
85     print("A renvoyé état b")
86     x_point_c = etat_retour(traj_c, systeme)[2]
87     print("A renvoyé état c")
88     print("Précision actuelle : ", abs(x_point_c))
89     while abs(x_point_c) > precision:
90         print()
91         print()
92         if x_point_c * x_point_a < 0:
93             bracket[1] = c
94             x_point_b = x_point_c
95         else:
96             bracket[0] = c
97             x_point_a = x_point_c
98
99         c = (bracket[0]+bracket[1])/2
100        gamma_c[3] = c
101        traj_c = rk4.simuler_trajectoire(gamma_c, 2*T/3, 100, systeme)
102        x_point_c = etat_retour(traj_c, systeme)[2]
103        print("Précision actuelle : ", abs(x_point_c))
104    return c

```

Il est important de mentionner que le code est imparfait : il nécessite parfois une intervention de l'utilisateur pour trouver un intervalle correct. Pour parfaire ce système, il serait potentiellement intéressant de parcourir l'orbite pendant $2T/3$ et de marquer l'instant de premier retour ainsi. Cette méthode n'a pas été mise en place dû à un manque de temps et d'utilité dans le cadre de cette étude (qui ne cherche qu'à montrer une capacité de calculer numériquement les variétés d'une orbite).

4.8 Calcul du linéarisé en tout point d'une trajectoire

Ce code en particulier viendra en fait servir celui qui calcule la matrice de monodromie, ainsi que ses objets propres. C'est ces objets précisément qui sont au coeur de toute l'étude menée dans cette présentation.

```
1 import src.superposition_potentielle as po
2 import numpy as np
3 import src.dichotomie as dich
4 G = 6.674e-11 # Constante gravitationnelle en SI
5
6 def r1(x,y,systeme):
7     return np.sqrt((x+systeme["barycentre"])**2 + y**2)
8
9 def r2(x,y,systeme):
10    return np.sqrt((x-(systeme["rayon"]-systeme["barycentre"]))**2 + y**2)
11
12 def Ubar_xx_sans_m(x,y,systeme):
13     terme1 = -po.omega(systeme)**2
14     terme2 = G*systeme["corps1"]["masse"]*((r1(x,y,systeme))**2-3*(x+
15         systeme["barycentre"])**2)/(r1(x,y,systeme)**5)
16     terme3 = G*systeme["corps2"]["masse"]*((r2(x,y,systeme))**2-3*(x-
17         systeme["rayon"]+systeme["barycentre"])**2)/(r2(x,y,systeme)**5)
18     return terme1 + terme2 + terme3
19
20 def Ubar_xy_sans_m(x,y,systeme):
21     terme1 = (systeme["corps1"]["masse"]*(x+systeme["barycentre"]))*y/(r1(
22         x,y,systeme)**5)
23     terme2 = (systeme["corps2"]["masse"]*(x-systeme["rayon"]+systeme["
24         barycentre"]))*y/(r2(x,y,systeme)**5)
25     return -3*G*(terme1 + terme2)
26
27 def Ubar_yy_sans_m(x,y,systeme):
28     terme1 = -po.omega(systeme)**2
29     terme2 = G*systeme["corps1"]["masse"]*((r1(x,y,systeme))**2-3*y**2)/(
30         r1(x,y,systeme)**5)
31     terme3 = G*systeme["corps2"]["masse"]*((r2(x,y,systeme))**2-3*y**2)/(
32         r2(x,y,systeme)**5)
33     return terme1 + terme2 + terme3
34
35 # N.B. : orbit is a 4D vector of the form [x,y,vx,vy]
36 # renvoie A_state le linéarisé en la position state
37 def systeme_linearise(state,systeme_actif):
38     x,y,vx,vy = state
39     return np.array([
40         [0,0,1,0],
41         [0,0,0,1],
```

```

37         [(-1)*Ubar_xx_sans_m(x,y,systeme_actif),(-1)*Ubar_xy_sans_m(x,y,
38             systeme_actif),0,2*po.omega(systeme_actif)],
39         [(-1)*Ubar_xy_sans_m(x,y,systeme_actif),(-1)*Ubar_yy_sans_m(x,y,
40             systeme_actif),-2*po.omega(systeme_actif),0]
41     ])
42
43 # derivee : renvoie la liste des A_i où A_i est le linéarisé en la
44 # position gamma_i
45 def derivee(orbit,systeme):
46     linearized_map = []
47     for i in range(len(orbit)):
48         linearized_map.append(systeme_linearise(orbit[i],systeme))
49     return linearized_map
50
51
52 def valeurs_propres_L1(systeme):
53     x_L1 = dich.dichotomie(0, systeme["rayon"], precision=1, systeme=
54         systeme, pas=10)
55     A = systeme_linearise([x_L1, 0, 0, 0],systeme)
56     return np.linalg.eigvals(A)
57
58 def tau(systeme,nu):
59     x_L1 = dich.dichotomie(0, systeme["rayon"], precision=1, systeme=
60         systeme, pas=10)
61     return (-1)*(nu**2 - Ubar_xx_sans_m(x_L1,0,systeme))/(2*po.omega(
62         systeme) * nu)
63
64 def sigma(systeme, lambdavp):
65     x_L1 = dich.dichotomie(0, systeme["rayon"], precision=1, systeme=
66         systeme, pas=10)
67     return (-1)*(lambdavp**2 - Ubar_xx_sans_m(x_L1,0,systeme))/(2*po.omega
68         (systeme) * lambdavp)

```


4.9 Calcul de la matrice de monodromie et obtention des éléments propres

Des codes de formatage L^AT_EX sont présents ici pour faciliter la recopie dans la présentation des valeurs numériques (sans quoi cela serait très long).

```
1 import src.rk4 as rk4
2 import src.dichotomie as dich
3 import src.lineariser as lin
4
5 import numpy as np
6
7 def matrice_monodromie(systeme, gamma, dt):
8     A = lin.derivee(gamma, systeme)
9     Phi = np.eye(4)
10
11     for n in range(len(A)-1):
12
13         A1 = A[n]
14         A2 = A[n]           # better: interpolate between A[n] and A[n+1]
15         A3 = A[n]
16         A4 = A[n+1]
17
18         k1 = A1 @ Phi
19         k2 = A2 @ (Phi + dt*k1/2)
20         k3 = A3 @ (Phi + dt*k2/2)
21         k4 = A4 @ (Phi + dt*k3)
22
23         Phi = Phi + dt*(k1 + 2*k2 + 2*k3 + k4)/6
24
25     return Phi
26
27
28
29 def latex_sci_reel(x, digits=2):
30     mantissa, exponent = f"{x:.{digits}e}".split("e")
31     exponent = int(exponent)
32
33     if exponent == 0:
34         return mantissa
35
36     return rf"{mantissa} \cdot 10^{{{exponent}}}"
37
38
39 def latex_sci(z, digits=2, tol=1e-12):
40     z = complex(z)
41
42     if abs(z.imag) < tol:
43         return latex_sci_reel(z.real, digits)
```

```

44
45     if abs(z.real) < tol:
46         return latex_sci_reel(z.imag, digits) + r"\mathrm{i}"
47
48     sign = "+" if z.imag >= 0 else "-"
49     return (
50         latex_sci_reel(z.real, digits)
51         + f" {sign} "
52         + latex_sci_reel(abs(z.imag), digits)
53         + r"\mathrm{i}"
54     )
55
56
57
58 def matrice_python_vers_bmatrix(matrix):
59     n = len(matrix)
60     p = len(matrix[0])
61     latex_code = "\\begin{bmatrix} \n"
62     for i in range(n):
63         for j in range(p):
64             latex_code += latex_sci(matrix[i][j])
65             if j != p-1:
66                 latex_code += " & "
67             latex_code += "\\ \\ \\ \n"
68     latex_code += "\\end{bmatrix}"
69     return latex_code
70
71
72 def latex_formater_valeurs_propres(M):
73     eigM = np.linalg.eig(M)
74     valpM = eigM[0]
75     vecpM = eigM[1]
76
77     latex_matrice_monodromie = matrice_python_vers_bmatrix(M)
78     latex_matrice_vecp = matrice_python_vers_bmatrix(vecpM)
79     print(latex_matrice_monodromie)
80     print(latex_matrice_vecp)
81
82     latex_valp = "\{"
83     for i in range(len(valpM)):
84         latex_valp += latex_sci(valpM[i])
85         if i != len(valpM)-1:
86             latex_valp += "\: , "
87     latex_valp += "\}"
88     print(latex_valp)

```

Calculs détaillés

Calcul pour L1

On a

$$\omega^2 x - \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en remplaçant ω^2 par son expression on obtient

$$\frac{G(M_1 + M_2)}{R^3} x - \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en reconnaissant $\mu = \frac{M_2}{M_1 + M_2}$ on peut simplifier en

$$\frac{x}{R^3} - \frac{1 - \mu}{(x + \mu R)^2} + \frac{\mu}{(x - (1 - \mu)R)^2} = 0$$

et en utilisant le fait que $x = \varepsilon + (1 - \mu)R$ on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{(\varepsilon + R)^2} + \frac{\mu}{\varepsilon^2} = 0$$

On peut alors procéder à un développement limité en $\frac{\varepsilon}{R}$. Ce dernier est ici supposé, motivé par le fait que l'on suppose $\mu \rightarrow 0$ ce qui nous a permis de, qualitativement, supposer $\varepsilon \rightarrow 0$. Le résultat final permettra, ou non, de légitimiser notre supposition. On obtient ainsi

$$\frac{1 - \mu}{(\varepsilon + R)^2} = \frac{1 - \mu}{R^2} \left(1 + \frac{\varepsilon}{R}\right)^{-2} \underset{\varepsilon \rightarrow 0}{=} \frac{1 - \mu}{R^2} \left(1 - \frac{2\varepsilon}{R} + O\left(\frac{\varepsilon^2}{R^2}\right)\right) \underset{\varepsilon \rightarrow 0}{=} \frac{1 - \mu}{R^2} - \frac{2(1 - \mu)\varepsilon}{R^3} + O\left(\frac{\varepsilon^2}{R^4}\right)$$

et en remplaçant dans l'expression que l'on avait avant on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{R^2} + \frac{2(1 - \mu)\varepsilon}{R^3} + \frac{\mu}{\varepsilon^2} + O\left(\frac{\varepsilon^2}{R^4}\right) = 0$$

ce qui se simplifie en

$$\frac{\varepsilon}{R^3} + \frac{(2 - 2\mu)\varepsilon}{R^3} + \frac{\mu}{\varepsilon^2} + O\left(\frac{\varepsilon^2}{R^4}\right) = \frac{3 - 2\mu}{R^3} \varepsilon + \frac{\mu}{\varepsilon^2} + O\left(\frac{\varepsilon^2}{R^4}\right) = 0$$

et donc à l'ordre 1, cela devient

$$\varepsilon^3 = -\frac{\mu}{3 - 2\mu} R^3 \underset{\mu \rightarrow 0}{\sim} -\frac{\mu}{3} R^3$$

et finalement

$$\varepsilon = -\left(\frac{\mu}{3}\right)^{\frac{1}{3}} R$$

Ainsi, notre DL en $\frac{\varepsilon}{R}$ est légitime car on observe que $\varepsilon = O_{\mu \rightarrow 0}(\mu^{\frac{1}{3}})$, R étant une constante.

Calcul pour L2¹

On a

$$\omega^2 x - \frac{GM_1}{(x + \mu R)^2} - \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en remplaçant ω^2 par son expression on obtient

$$\frac{G(M_1 + M_2)}{R^3} x - \frac{GM_1}{(x + \mu R)^2} - \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en reconnaissant $\mu = \frac{M_2}{M_1 + M_2}$ on peut simplifier en

$$\frac{x}{R^3} - \frac{1 - \mu}{(x + \mu R)^2} - \frac{\mu}{(x - (1 - \mu)R)^2} = 0$$

et en utilisant le fait que $x = \varepsilon + (1 - \mu)R$ on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{(\varepsilon + R)^2} - \frac{\mu}{\varepsilon^2} = 0$$

On peut alors procéder à un développement limité en $\frac{\varepsilon}{R}$. Ce dernier est ici supposé, motivé par le fait que l'on suppose $\mu \rightarrow 0$ ce qui nous a permis de, qualitativement, supposer $\varepsilon \rightarrow 0$. Le résultat final permettra, ou non, de légitimiser notre supposition. On obtient ainsi

$$\frac{1 - \mu}{(\varepsilon + R)^2} = \frac{1 - \mu}{R^2} \left(1 + \frac{\varepsilon}{R}\right)^{-2} \underset{\varepsilon \rightarrow 0}{=} \frac{1 - \mu}{R^2} \left(1 - \frac{2\varepsilon}{R} + O\left(\frac{\varepsilon^2}{R^2}\right)\right) \underset{\varepsilon \rightarrow 0}{=} \frac{1 - \mu}{R^2} - \frac{2(1 - \mu)\varepsilon}{R^3} + O\left(\frac{\varepsilon^2}{R^4}\right)$$

et en remplaçant dans l'expression que l'on avait avant on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{R^2} + \frac{2(1 - \mu)\varepsilon}{R^3} - \frac{\mu}{\varepsilon^2} + O\left(\frac{\varepsilon^2}{R^4}\right) = 0$$

ce qui se simplifie en

$$\frac{\varepsilon}{R^3} + \frac{(2 - 2\mu)\varepsilon}{R^3} - \frac{\mu}{\varepsilon^2} + O(\varepsilon^2) = \frac{3 - 2\mu}{R^3} \varepsilon - \frac{\mu}{\varepsilon^2} + O(\varepsilon^2) = 0$$

et donc à l'ordre 1, cela devient

$$\varepsilon^3 = \frac{\mu}{3 - 2\mu} R^3 \underset{\mu \rightarrow 0}{\sim} \frac{\mu}{3} R^3$$

et finalement

$$\varepsilon = \left(\frac{\mu}{3}\right)^{\frac{1}{3}} R$$

Ainsi, notre DL en $\frac{\varepsilon}{R}$ est légitime car on observe que $\varepsilon = O(\mu^{\frac{1}{3}})$, R étant une constante.

1. On pourra remarquer que les calculs sont identiques à ceux de L1 au signe près du terme en $\frac{\mu}{\varepsilon^2}$.

Calcul pour L3

On a

$$\omega^2 x + \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en remplaçant ω^2 par son expression on obtient

$$\frac{G(M_1 + M_2)}{R^3} x + \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en reconnaissant $\mu = \frac{M_2}{M_1 + M_2}$ on peut simplifier en

$$\frac{x}{R^3} + \frac{1 - \mu}{(x + \mu R)^2} + \frac{\mu}{(x - (1 - \mu)R)^2} = 0$$

et en utilisant le fait que $x = \delta - R$ on obtient

$$\frac{\delta}{R^3} - \frac{1}{R^2} + \frac{1 - \mu}{(\delta - (1 - \mu)R)^2} + \frac{\mu}{(\delta - (2 - \mu)R)^2} = 0$$

On peut alors procéder à deux développements limités en $w := \mu + \frac{\delta}{R}$. Ces derniers sont supposés, motivée par le fait que l'on suppose $\mu \rightarrow 0$ ce qui nous a permis de, qualitativement, supposer $\delta \rightarrow 0$ et donc $w \rightarrow 0$. Le résultat final permettra ici aussi de légitimiser, ou non, notre supposition. On obtient alors

$$\frac{1 - \mu}{(\delta - (1 - \mu)R)^2} = \frac{1 - \mu}{R^2} (1 - w)^{-2} \underset{w \rightarrow 0}{=} \frac{1 - \mu}{R^2} (1 + 2w + o(w)) \underset{w \rightarrow 0}{=} \frac{1 - \mu}{R^2} + \frac{2(1 - \mu)w}{R^2} + o(w)$$

or on a $(1 - \mu)w = w - \mu w \underset{\mu \rightarrow 0}{=} w + o(w)$ ce qui simplifie le terme précédent en

$$\frac{1 - \mu}{R^2} + \frac{2w}{R^2} + \underset{w \rightarrow 0}{o}(w) = \frac{1 + \mu + 2\frac{\delta}{R}}{R^2} + \underset{w \rightarrow 0}{o}(w)$$

On calcule de même

$$\frac{\mu}{(\delta - (2 - \mu)R)^2} = \frac{\mu}{4R^2} (1 - \frac{1}{2}w)^{-2} \underset{w \rightarrow 0}{=} \frac{\mu}{4R^2} (1 + w + o(w)) \underset{w \rightarrow 0}{=} \frac{\mu}{4R^2} + \frac{\mu w}{4R^2} + o(w) \underset{w \rightarrow 0}{=} \frac{\mu}{4R^2} + o(w)$$

On peut remplacer ces deux expressions dans l'expression originale, et obtenir ainsi

$$\frac{\delta}{R^3} - \frac{1}{R^2} + \frac{1 + \mu + 2\delta}{R^2} + \underset{w \rightarrow 0}{o}(w) + \frac{\mu}{4R^2} + \underset{w \rightarrow 0}{o}(w) = \frac{3\delta}{R^3} + \frac{5\mu}{4R^2} + \underset{w \rightarrow 0}{o}(w) = 0$$

ce qui devient à l'ordre 1

$$\frac{\delta}{R^3} = -\frac{5\mu}{12R^2}$$

et donc

$$\delta = -\frac{5\mu}{12}R$$

On peut alors constater que $o(w) = o(\mu + \frac{\delta}{R}) = o(\mu - \frac{5\mu}{12}) = o(\mu)$ ou aussi $o(w) = o(\delta)$. On vérifie ainsi que l'ordre 1 que l'on a écrit est bien celui en δ .

Calcul pour L4 et L5

On souhaite d'abord démontrer le passage des relations (15) aux relations (16). On utilise alors $(\frac{1}{u})' = \frac{u'}{u^2}$. On sépare chaque cas suivant si l'on dérive par rapport à x , où y .

Cas de dérivation selon x :

On a dans ce cas $f_1 : (x, y) \mapsto \frac{GmM_1}{r_1} = \frac{GmM_1}{\sqrt{(x+\mu R)^2 + y^2}}$ et $f_2 : (x, y) \mapsto \frac{GmM_2}{r_2}$, toutes deux des fonctions dérivables selon x sur $\mathbb{R}^*$ comme inverse de fonctions dérivables selon x , à savoir r_1 et r_2 . Ces dernières sont de dérivées partielles selon x

$$\begin{cases} \frac{\partial r_1}{\partial x}(x, y) = \frac{2(x+\mu R)}{2\sqrt{(x+\mu R)^2 + y^2}} = \frac{(x+\mu R)}{r_1(x, y)} \\ \frac{\partial r_2}{\partial x}(x, y) = \frac{(x-(1-\mu)R)}{r_2(x, y)} \end{cases}$$

ce qui nous donne les expressions des dérivées partielles de f_1 et f_2 suivantes (on utilisera la simple notation f' par souci de lisibilité, mais on a bien à faire à des dérivées partielles, le contexte indiquant clairement que c'est selon x ici) :

$$\begin{cases} f'_1(x, y) = \frac{GmM_1 r'_1(x, y)}{r_1^2(x, y)} = \frac{GmM_1}{r_1^2(x, y)} \times \frac{(x+\mu R)}{r_1(x, y)} = \frac{GmM_1(x+\mu R)}{r_1^3(x, y)} \\ f'_2(x, y) = \frac{GmM_2 r'_2(x, y)}{r_2^2(x, y)} = \frac{GmM_2(x-(1-\mu)R)}{r_2^3(x, y)} \end{cases}$$

et assez trivialement, $x \mapsto -\frac{1}{2}m\omega^2(x^2 + y^2)$ admet $x \mapsto -m\omega^2 x$ pour dérivée. On obtient ainsi exactement le terme pour $\frac{\partial \bar{U}}{\partial x}$ en sommant chacune des dérivées. Quand au cas selon y , on peut appliquer exactement le même principe. Les calculs seront même plus simple car on voit assez clairement que dans ce cas, $\frac{\partial r_1}{\partial y}(x, y) = \frac{y}{r_1(x, y)}$. Et on obtient bien ainsi les relations (16).

Explicitons le passage de la relation (16) à la relation (17). On commence par imposer la condition d'équilibre, et remplacer par son expression $\omega^2 = \frac{G(M_1+M_2)}{R^3}$:

$$\begin{cases} 0 = -\frac{Gm(M_1+M_2)}{R^3}x + \frac{GmM_1(x+\mu R)}{r_1^3} + \frac{GmM_2(x-(1-\mu)R)}{r_2^3} \\ 0 = -\frac{Gm(M_1+M_2)}{R^3}y + \frac{GmM_1 y}{r_1^3} + \frac{GmM_2 y}{r_2^3} \end{cases}$$

ce qui se simplifie en

$$\begin{cases} \frac{M_1+M_2}{R^3}x = \frac{M_1(x+\mu R)}{r_1^3} + \frac{M_2(x-(1-\mu)R)}{r_2^3} \\ \frac{M_1+M_2}{R^3} = \frac{M_1}{r_1^3} + \frac{M_2}{r_2^3} \end{cases}$$

On peut alors diviser par $M_1 + M_2$. On reconnaît alors deux termes : $1 - \mu = \frac{M_1}{M_1+M_2}$ et $\mu = \frac{M_2}{M_1+M_2}$ ce qui donne alors

$$\begin{cases} \frac{x}{R^3} = \frac{(1-\mu)(x+\mu R)}{r_1^3} + \frac{\mu(x-(1-\mu)R)}{r_2^3} \\ \frac{1}{R^3} = \frac{(1-\mu)}{r_1^3} + \frac{\mu}{r_2^3} \end{cases}$$

et on peut simplifier encore plus la première expression de manière à faire apparaître la deuxième relation, donnant alors

$$\frac{x}{R^3} = \left(\frac{1-\mu}{r_1^3} + \frac{\mu}{r_2^3}\right)x + \frac{\mu(1-\mu)R}{r_1^3} - \frac{\mu(1-\mu)R}{r_2^3}$$

ce qui est le résultat voulu.

Explicitons le passage des relations (17) à la relation (18).

$$\frac{x}{R^3} = \frac{x}{R^3} + \mu(1 - \mu)\left(\frac{1}{r_1^3} - \frac{1}{r_2^3}\right)R \quad \text{i.e.} \quad \frac{1}{r_1^3} - \frac{1}{r_2^3} = 0$$

ce qui, dans $\mathbb{R}$, nous donne

$$r_1 = r_2$$

et en remplaçant cela dans la deuxième relation de (17) on obtient aussi

$$\frac{1}{R^3} = \frac{1 - \mu}{r_1^3} + \frac{\mu}{r_1^3} = \frac{1}{r_1^3}$$

et on a alors dans $\mathbb{R}$ que

$$R = r_1 = r_2$$

d'où la relation (18).